Bölüm 7: Çoklu Ajan Sistemleri

Algoritmik Oyun Kuramı

Ajan özellikleri | İşbirliği ve rekabet | Merkezi mi dağıtık mı | Sözleşme ağı | Koalisyon oyunları | Shapley değeri | Çekirdek | Güç endeksleri | Kamu malı ve beleşçilik

Sercan KÜLCÜ · Giresun Üniversitesi · Bilgisayar Mühendisliği

Öğrenme Çıktıları

Ajan ve MAS

Ajanın temel özelliklerini ve çoklu ajan sistemlerinde işbirliği, rekabet ve koordinasyon sorunlarını açıklamak.

Görev tahsisi

Sözleşme ağı protokolünü (duyuru, teklif, ödül) küçük bir örnek üzerinde çalıştırmak ve optimumla karşılaştırmak.

Koalisyon oyunları

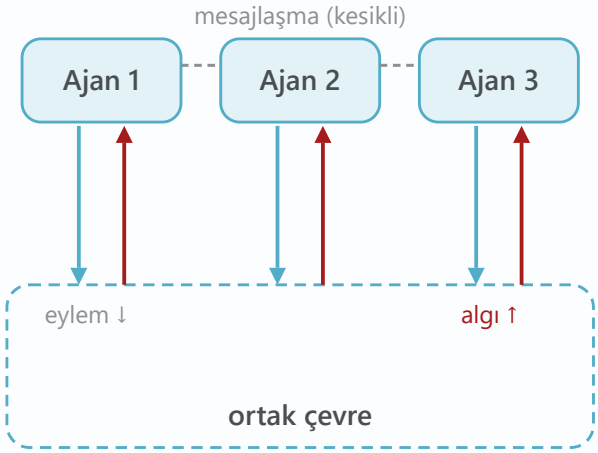
Karakteristik fonksiyonu yazmak; Shapley değerini tüm sıralamalardaki marjinal katkılardan hesaplamak; çekirdeği sınamak; ağırlıklı oylamada güç endeksini bulmak.

Kamu malı ve beleşçilik

Kamu malı oyununda katkı vermemenin neden baskın strateji olduğunu sayısal olarak göstermek ve çözüm yollarını tartışmak.

Çoklu Ajan Sistemi (Multi-Agent System, MAS) Nedir?

- Aynı çevrede **algılayan, karar veren ve eylem yapan** birden fazla otonom ajandan oluşan sistem.
- Ajanlar ortak hedefe sahip olabilir ya da kendi çıkarları doğrultusunda **rekabet** edebilir.
- Her ajanın bilgisi **yereldir**; bu yüzden koordinasyon ve iletişim temel tasarım sorunlarıdır.
- Örnekler: robot takımları, otonom araç filoları, dağıtık sensör ağları, trafik ve ekonomi simülasyonları.



Oyun kuramı bağlantısı: ajanlar birbirinin kararından etkileniyorsa etkileşim bir **oyundur**; ajanları tasarlayan kişi ise kuralları koyan **mekanizma tasarımcısıdır**.

Ajan Kavramı ve Özellikleri

Wooldridge ve Jennings'in (1995) "zayıf ajan" tanımındaki dört özellik:

Otonomi (autonomy)
İnsan müdahalesi olmadan kendi kararını verir, kendi durumunu yönetir.
`drone rotasını kendisi seçer`

Reaktivite (reactivity)
Çevredeki değişikliği algılar ve zamanında tepki verir.
`engel görünce frenler`

Proaktivite (pro-activeness)
Yalnızca tepki vermez; hedefe ulaşmak için inisiyatif alır, plan yapar.
`şarjı bitmeden istasyona gider`

Sosyal yetenek (social ability)
Diğer ajanlarla iletişim kurar, pazarlık eder, işbirliği yapar.
`görev için teklif verir`

Oyun kuramında ajan = **rasyonel oyuncu**: tercihleri bir fayda fonksiyonuyla ifade edilir ve beklenen faydasını en büyüklemeye çalışır.

Ajan Nasıl Karar Verir? Mimariler

Mimari	Karar mekanizması	Güçlü yanı	Örnek
Reaktif	Durum → eylem kuralları; iç model yok	Hızlı, sağlam	Süpürge robotu, sürü davranışı
Müzakereci (deliberative)	Çevre modeli + planlama / arama	Uzun vadeli hedefler	Keşif robotu, lojistik planlayıcı
BDI	İnanç (belief), arzu (desire), niyet (intention)	İnsan benzeri akıl yürütme	Hava trafik yönetimi, sanal asistan
Öğrenen	Deneyimden politika öğrenir	Bilinmeyen çevreye uyum	Oyun oynayan ajanlar
Hibrit	Reaktif alt katman + planlayıcı üst katman	Hız ve öngörü birlikte	Otonom araç

Mimari, ajanın içini anlatır; oyun kuramı ise ajanlar arasındaki **etkileşimi** inceler.

Gerçek Hayat Uygulamaları

Otonom araçlar
Kavşakta geçiş sırası, şerit değiştirme ve konvoy sürüşü için koordinasyon.

Drone takımları
Arama-kurtarmada bölgeleri paylaşma: görev tahsisi problemi.

Depo robotları
Yüzlerce robotun sipariş toplaması; çarpışmasız rota ve iş dağıtımı.

Çevrimiçi pazar yerleri
Alıcı ve satıcı ajanlar, fiyat teklifleri ve otomatik pazarlık.

Akıllı şebeke
Evler, depolar ve üreticiler arasında enerji alışverişi; talep yönetimi.

Simülasyon
Trafik, salgın ve ekonomi modellerinde bireylerin ajan olarak canlandırılması.

Ortak soru: kendi hedefi olan ajanlar bir araya geldiğinde **sistem düzeyinde** iyi bir sonuç nasıl elde edilir?

İşbirliği, Rekabet ve Karma Sistemler

İşbirlikçi MAS

Ortak hedef: ajanlar bilgi, görev ve kaynak paylaşır. Sorun *koordinasyon*dur.

`arama-kurtarma drone takımı`

Rekabetçi MAS

Amaçlar farklı: her ajan diğerlerinin davranışını hesaba katar. Sorun *strateji*dir.

`açık artırmada teklifçiler`

Karma (mixed-motive)

Bazı hedefler ortak, bazı kaynaklar rekabet konusu. Gerçek sistemlerin çoğu buradadır.

`aynı şebekeyi paylaşan evler`

Hangisi daha verimli?	Koşul
İşbirliği	Hedefler örtüşüyor, iletişim ucuz, tek ajanın yapamayacağı iş var
Rekabet	Özel bilgi var ve fiyat/teklif ile açığa çıkarılabilir (piyasa, açık artırma)

Yanıt, sistemin amacına ve kaynak yapısına bağlıdır; mekanizma tasarımı rekabeti iyi sonuca yönlendirmeye çalışır.

Çıkar Yelpazesi: Üç Küçük Oyun



Saf koordinasyon
ortak çıkar: $u_1 = u_2$

	Sağ	Sol
Sağ	(2, 2)	(0, 0)
Sol	(0, 0)	(1, 1)

iki saf denge: sorun hangisi?

Mahkûmun İkilemi
karma çıkar

	Sessiz	İhanet
Sessiz	(−1, −1)	(−3, 0)
İhanet	(0, −3)	(−2, −2)

tek denge (−2, −2), verimsiz

Yazı-Tura Eşleştirme
saf rekabet: $u_1 + u_2 = 0$

	Yazı	Tura
Yazı	(1, −1)	(−1, 1)
Tura	(−1, 1)	(1, −1)

saf denge yok; karışık (½, ½)

- Solda çıkarlar **aynı**: iletişim ve konvansiyon yeter (koordinasyon sorunu).
- Ortada çıkarlar **kısmen çatışır**: bireysel teşvik ortak iyiye ters düşer, teşvik tasarımı gerekir.
- Sağda biri kazanırken öteki kaybeder (sıfır toplamlı): öngörülemez olmak, yani **karışık strateji** (mixed strategy) önemlidir.

Turuncu hücreler: saf Nash dengeleri (her iki oyuncu da en iyi tepkide).

Bireysel Rasyonellik ve Ortak Kötü Sonuç

satır ╲ sütun	Sessiz	İhanet
Sessiz	(−1, −1)	(−3, 0)
İhanet	(0, −3)	(−2, −2)

Mahkûmun İkilemi: işbirliği = Sessiz, bencil seçim = İhanet

- İhanet **baskın strateji**: karşı taraf ne yaparsa yapsın daha az ceza.
- Denge (−2, −2), oysa (−1, −1) ikisi için de daha iyi: bireysel rasyonellik **ortak kötü sonuç** doğurur.
- MAS'te aynı yapı: bant genişliği paylaşımı, ortak kaynak kullanımı, bilgi paylaşımı.

Çok oyunculu sürümü **kamu malı oyunudur**; ortak kaynağın tükenmesi ise ortak mallar trajedisidir.

Karar Alma: Merkezi mi, Dağıtık mı?

Ajanlar çevresel bilgiye göre plan yapar. Kararı kim verir: tek bir merkez mi, her ajan kendisi mi?

	Merkezi kontrol	**Dağıtık (tam otonomi)**
Avantaj	Tüm bilgi tek noktada: **optimum** hesaplanabilir; kolay koordinasyon, düşük çatışma riski	Esnek, ölçeklenebilir, yerel olaylara hızlı tepki; bir parça bozulunca sistem çalışmaya devam eder
Dezavantaj	Tek nokta hatası ve darboğaz; ajan sayısı artınca hesaplama patlar; ajanlar özel bilgisini dürüst bildirmeyebilir	Yerel bilgiyle alınan kararlar uyumsuz olabilir; bencil ajanlar verimsiz dengeye gidebilir (Anarşinin Bedeli)
Örnek	Trafik ışığı yönetim merkezi; sensör ağında merkezî sunucu	Drone takımları, otonom araçlar, robotik keşif

Çoğu gerçek MAS **hibrittir**: ajanlar otonom karar verir, merkez kuralları, fiyatları ya da ihaleleri belirler. Sözleşme ağı buna bir örnektir.

Dağıtık Koordinasyon ve Konvansiyonlar

- Ajanlar merkez olmadan **yerel bilgi ve mesajlaşma** ile karar verir.
- Protokol; iletişim gecikmesi, hata toleransı ve ölçeklenebilirliği birlikte ele almalıdır.
- Ortak bilgi eksikliği uyumsuz planlara yol açar: iki robot dar koridorda karşılaşınca ikisi de aynı tarafa kaçabilir.
- Çözüm: **konvansiyon / sosyal yasa** ("herkes sağdan") — koordinasyon oyununda bir dengeyi önceden seçer.

R1 ╲ R2	Sağ	Sol
Sağ	(1, 1)	(−1, −1)
Sol	(−1, −1)	(1, 1)

Koridor oyunu: iki saf Nash dengesi (turuncu)

İki denge de eşit iyidir; sorun hangisinin oynanacağıdır. Trafik ışığı da benzer biçimde bir **korelasyonlu denge** aracıdır.

MAS'te Oyun Kuramı ve Teşvikler

Analiz: Nash dengesi

Ajanların stratejik kararları dengede nereye varır? Denge verimli mi?

Tasarım: mekanizma

Kurallar, ödemeler ve açık artırmalar ajanları dürüst bildirime ve işbirliğine yönlendirir.

Enerji paylaşımı

Dengeli teşvik (ör. fazla üretime adil ödeme) → herkes yeterli enerji alır; yanlış teşvik → herkes pik saatte çeker.

Dağıtık ağ kaynakları

Bant genişliği ve hesaplama gücü: kullanım ücreti ya da kota olmazsa aşırı kullanım.

Doğru teşvik

+ İşbirliğini artırır, çatışmayı azaltır.

Yanlış teşvik

– Bencil davranış, aşırı risk ve beleşçilik doğurur.

İletişim, Öğrenme ve Ortaya Çıkan Davranış

İletişim ve bilgi

Ajanların bilgisi farklıdır; kimin neyi bildiği (bilgi yapısı) dengeyi değiştirir (eksik bilgili oyunlar). Ortak dil ve protokol gerekir: FIPA-ACL mesaj standardı, sözleşme ağını da bir etkileşim protokolü olarak tanımlar.

Öğrenme ve uyum

Ajanlar geri bildirimle stratejisini günceller: en iyi tepki dinamiği, kurgusal oyun, pekiştirmeli öğrenme. Diğer ajanlar da öğrendiği için her ajanın çevresi *durağan olmayan* bir çevredir; yakınsama garanti değildir.

Ortaya çıkan davranış

Basit yerel kurallar sistem düzeyinde beklenmedik örüntüler doğurur. Reynolds'ın (1987) "boids" modelinde üç kural (ayrılma, hizalanma, birleşme) sürü hareketini; Schelling'in (1971) modelinde hafif bir komşu tercihi belirgin mahalle ayrışmasını üretir.

Sistemin davranışı tek tek ajanlara bakarak okunamaz: denge analizi ile simülasyon birlikte kullanılır. Bölüm sonundaki kamu malı deneylerinde katkıların turdan tura düşmesi de ajanlar arası etkileşimden doğan bir örüntüdür.

Görev ve Kaynak Tahsisi (Task Allocation)

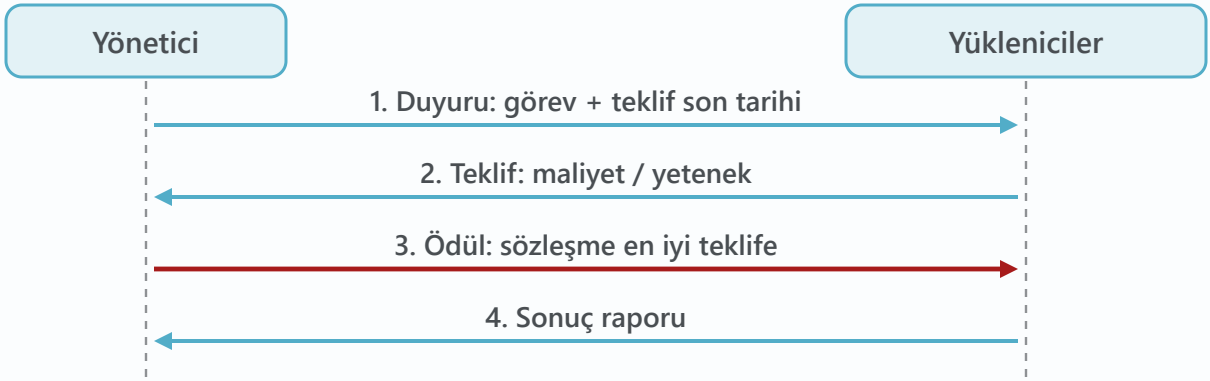
- Amaç: yetenek ve maliyetleri dikkate alarak işleri **uygun ajanlara** dağıtmak (ör. toplam süreyi en aza indirmek).
- Zorluk: maliyetleri çoğu zaman yalnızca ajanın kendisi bilir (**özel bilgi**); ajan sayısı büyüktür; görevler zamanla gelir.
- İyi bir yöntem çözüm kalitesini iletişim ve hesaplama maliyetiyle dengeler.

Yöntem	Nasıl çalışır?	Artı / eksi
Merkezi optimizasyon	Tüm maliyetler toplanır, atama problemi çözülür (Macar algoritması, $O(n^3)$)	Optimum / tüm bilgi gerekir
Sözleşme ağı	Görev duyurulur, ajanlar teklif verir, en iyi teklif kazanır	Basit, dağıtık / açgözlü olabilir
Açık artırma	Görevler/kaynaklar teklifle satılır; kombinatoryal açık artırmada paketler	Teşvik uyumlu tasarlanabilir
Pazar tabanlı	Fiyatlar arz–talebe göre ayarlanır, ajanlar fiyata tepki verir	Ölçeklenir / yakınsama yavaş olabilir

GÖREV TAHSİSİ

Sözleşme Ağı Protokolü (Contract Net Protocol)

Smith (1980): görevi olan ajan **yönetici** (manager), işi yapabilecek ajanlar **yüklenici** (contractor) rolündedir. Roller görev başına değişebilir.



Yönetici maliyetleri bilmez; teklifler bu **özel bilgiyi** açığa çıkarır. Teklif verme bir açık artırmadır: yüklenici yalan söylerse ne olur? (Vickrey)

GÖREV TAHSİSİ

Sözleşme Ağı Adım Adım: 3 Görev, 3 Robot



maliyet (dk) = teklif

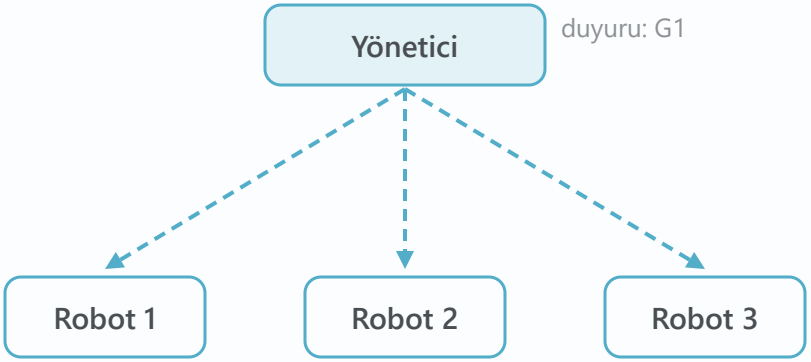
	R1	R2	R3
G1	2	3	6
G2	3	7	8
G3	5	4	9

Yönetici üç görevi (G1–G3) sırayla ihaleye çıkaracak. Her robot en çok bir görev alabilir; kendi maliyetini (tahmini süre) **yalnızca robot bilir** — yönetici tabloyu görmez.

Adım 1 / 11

Gösterim: kesikli ok = duyuru · turuncu = teklif · kalın kırmızı ok / çerçeve = ödül · soluk robot = meşgul · kesikli mavi çerçeve = optimum atama.

Sözleşme Ağı Adım Adım: 3 Görev, 3 Robot



maliyet (dk) = teklif

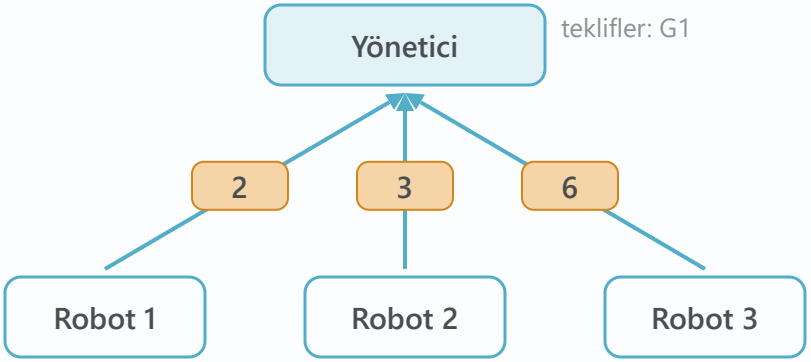
	R1	R2	R3
G1	2	3	6
G2	3	7	8
G3	5	4	9

toplam: 0 dk

Duyuru (task announcement): yönetici G1 görevini boştaki robotlara yayınlar: Robot 1, Robot 2, Robot 3.

Gösterim: kesikli ok = duyuru · turuncu = teklif · kalın kırmızı ok / çerçeve = ödül · soluk robot = meşgul · kesikli mavi çerçeve = optimum atama.

Sözleşme Ağı Adım Adım: 3 Görev, 3 Robot



maliyet (dk) = teklif

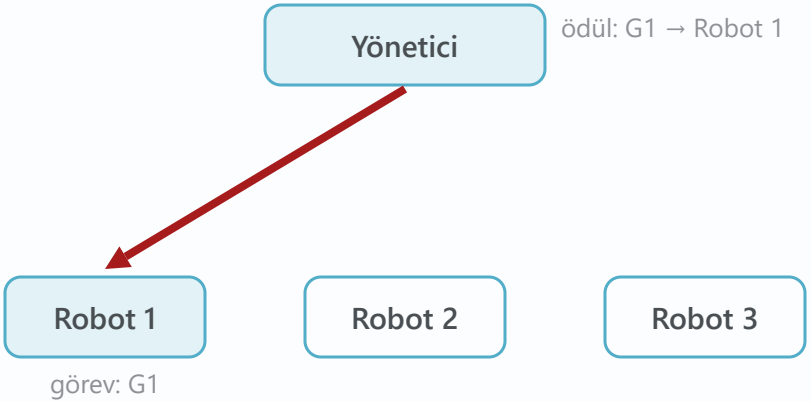
	R1	R2	R3
G1	2	3	6
G2	3	7	8
G3	5	4	9

toplam: 0 dk

Teklifler (bid): her boş robot G1 için kendi maliyetini bildirir: R1 → 2, R2 → 3, R3 → 6.

Gösterim: kesikli ok = duyuru · turuncu = teklif · kalın kırmızı ok / çerçeve = ödül · soluk robot = meşgul · kesikli mavi çerçeve = optimum atama.

Sözleşme Ağı Adım Adım: 3 Görev, 3 Robot



maliyet (dk) = teklif

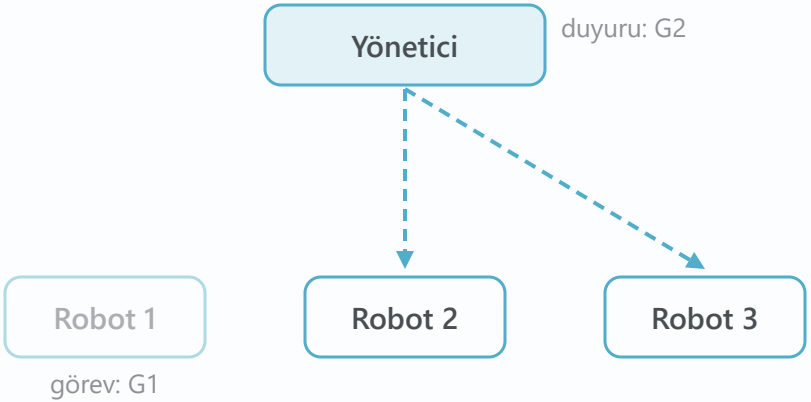
	R1	R2	R3
G1	2	3	6
G2	3	7	8
G3	5	4	9

toplam: 2 dk

Ödül (award): en düşük teklif Robot 1 (2 dk) — G1 ona verilir; robot artık meşgul.

Gösterim: kesikli ok = duyuru · turuncu = teklif · kalın kırmızı ok / çerçeve = ödül · soluk robot = meşgul · kesikli mavi çerçeve = optimum atama.

Sözleşme Ağı Adım Adım: 3 Görev, 3 Robot



maliyet (dk) = teklif

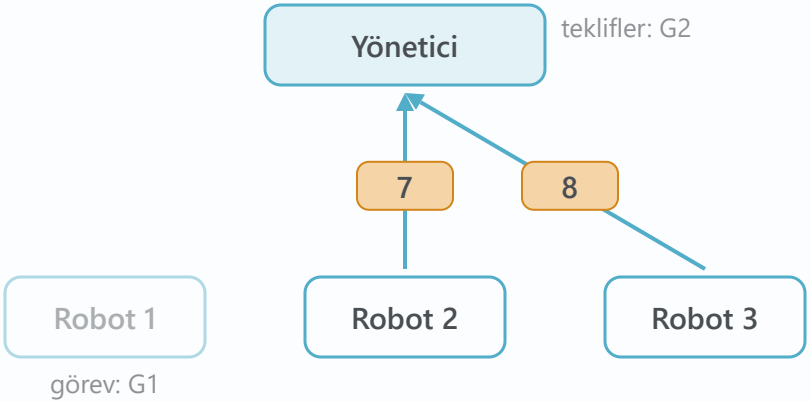
	R1	R2	R3
G1	2	3	6
G2	3	7	8
G3	5	4	9

toplam: 2 dk

Duyuru (task announcement): yönetici G2 görevini boştaki robotlara yayınlar: Robot 2, Robot 3.

Gösterim: kesikli ok = duyuru · turuncu = teklif · kalın kırmızı ok / çerçeve = ödül · soluk robot = meşgul · kesikli mavi çerçeve = optimum atama.

Sözleşme Ağı Adım Adım: 3 Görev, 3 Robot



maliyet (dk) = teklif

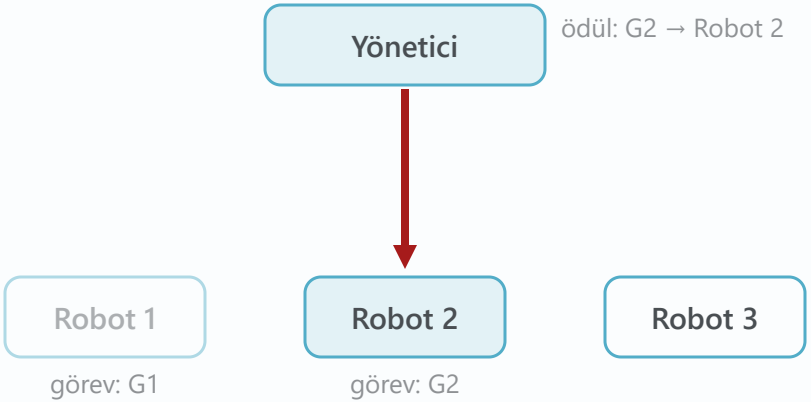
	R1	R2	R3
G1	2	3	6
G2	3	7	8
G3	5	4	9

toplam: 2 dk

Teklifler (bid): her boş robot G2 için kendi maliyetini bildirir: R2 → 7, R3 → 8.

Gösterim: kesikli ok = duyuru · turuncu = teklif · kalın kırmızı ok / çerçeve = ödül · soluk robot = meşgul · kesikli mavi çerçeve = optimum atama.

Sözleşme Ağı Adım Adım: 3 Görev, 3 Robot



maliyet (dk) = teklif

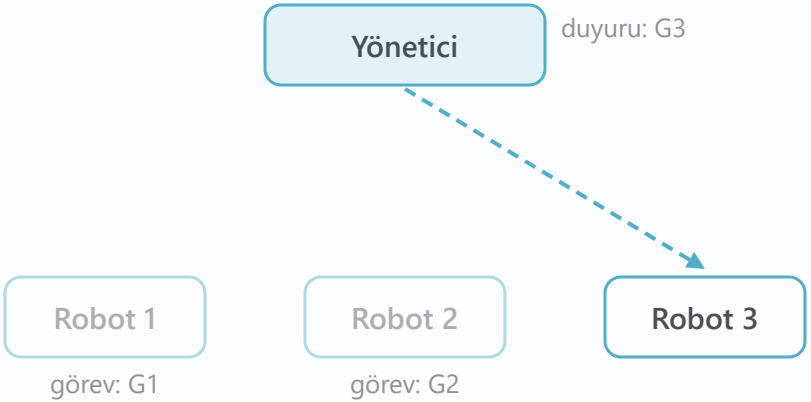
	R1	R2	R3
G1	2	3	6
G2	3	7	8
G3	5	4	9

toplam: 9 dk

Ödül (award): en düşük teklif Robot 2 (7 dk) — G2 ona verilir; robot artık meşgul.

Gösterim: kesikli ok = duyuru · turuncu = teklif · kalın kırmızı ok / çerçeve = ödül · soluk robot = meşgul · kesikli mavi çerçeve = optimum atama.

Sözleşme Ağı Adım Adım: 3 Görev, 3 Robot



maliyet (dk) = teklif

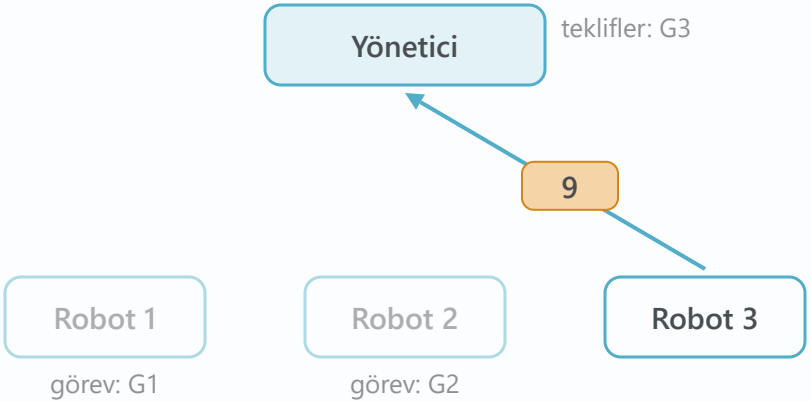
	R1	R2	R3
G1	2	3	6
G2	3	7	8
G3	5	4	9

toplam: 9 dk

Duyuru (task announcement): yönetici G3 görevini boştaki robotlara yayınlar: Robot 3.

Gösterim: kesikli ok = duyuru · turuncu = teklif · kalın kırmızı ok / çerçeve = ödül · soluk robot = meşgul · kesikli mavi çerçeve = optimum atama.

Sözleşme Ağı Adım Adım: 3 Görev, 3 Robot



maliyet (dk) = teklif

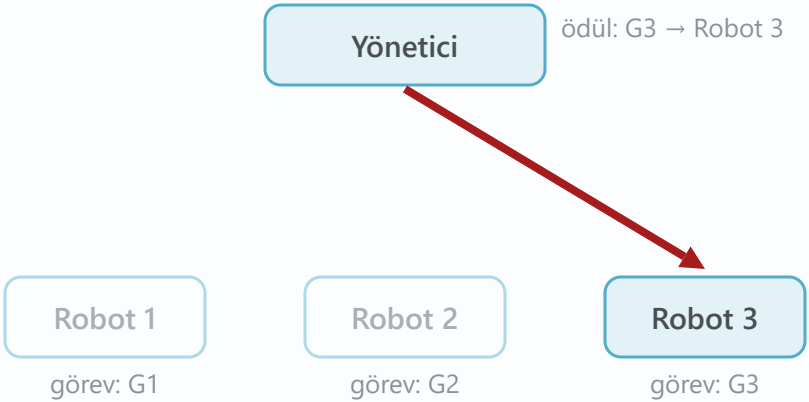
	R1	R2	R3
G1	2	3	6
G2	3	7	8
G3	5	4	9

toplam: 9 dk

Teklifler (bid): her boş robot G3 için kendi maliyetini bildirir: R3 → 9.

Gösterim: kesikli ok = duyuru · turuncu = teklif · kalın kırmızı ok / çerçeve = ödül · soluk robot = meşgul · kesikli mavi çerçeve = optimum atama.

Sözleşme Ağı Adım Adım: 3 Görev, 3 Robot



maliyet (dk) = teklif

	R1	R2	R3
G1	2	3	6
G2	3	7	8
G3	5	4	9

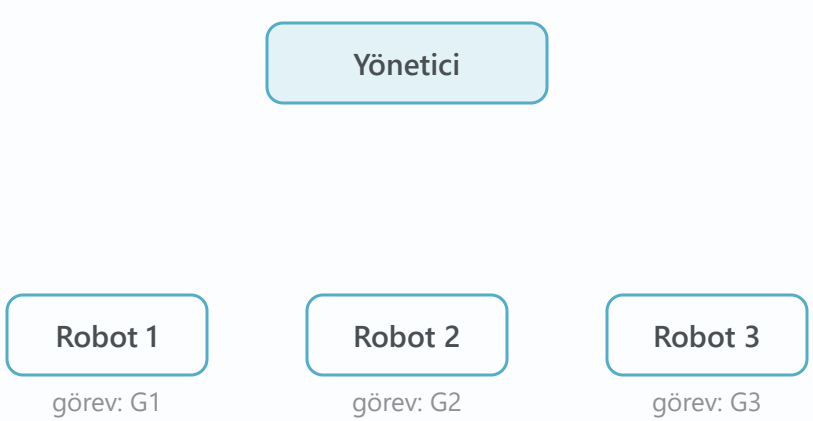
toplam: 18 dk

Ödül (award): en düşük teklif Robot 3 (9 dk) — G3 ona verilir; robot artık meşgul. Tek teklif kaldığı için seçim zorunlu.

Gösterim: kesikli ok = duyuru · turuncu = teklif · kalın kırmızı ok / çerçeve = ödül · soluk robot = meşgul · kesikli mavi çerçeve = optimum atama.

GÖREV TAHSİSİ

Sözleşme Ağı Adım Adım: 3 Görev, 3 Robot



maliyet (dk) = teklif

	R1	R2	R3
G1	2	3	6
G2	3	7	8
G3	5	4	9

sözleşme ağı: 18 dk

optimum: 13 dk

Sonuç: sözleşme ağı toplam **18 dk** (kırmızı). Tüm 6 atamayı deneyen optimum **13 dk** (kesikli mavi): G1→R3, G2→R1, G3→R2. Açgözlü sıralı ihale %38 daha pahalı.

Gösterim: kesikli ok = duyuru · turuncu = teklif · kalın kırmızı ok / çerçeve = ödül · soluk robot = meşgul · kesikli mavi çerçeve = optimum atama.

GÖREV TAHSİSİ

Kod İzleme: `sozlesme_agi(gorevler, maliyet)`

	R1	R2	R3
G1	2	3	6
G2	3	7	8
G3	5	4	9

maliyet[r][g] (dk) — yalnız robot bilir

atama

G1 → ?

G2 → ?

G3 → ?

toplam = 0 dk

gorevler = [G1, G2, G3] sırayla ihaleye çıkar. Tüm robotlar boş; atama henüz yok.

Adım 1 / 11

```
▶ def sozlesme_agi(gorevler, maliyet):
▶     bos = set(maliyet)
▶     atama, toplam = {}, 0
4     for g in gorevler:
5         teklif = {r: maliyet[r][g] for r in bos}
6         kazanan = min(teklif, key=teklif.get)
7         atama[g] = kazanan
8         toplam += teklif[kazanan]
9         bos.remove(kazanan)
10    return atama, toplam
```

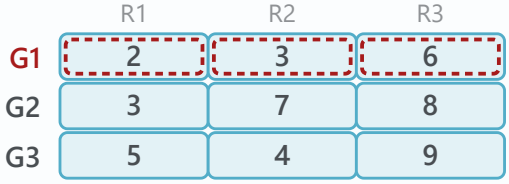
DEĞİŞKENLER

`bos = {R1, R2, R3}`

`atama = {}` `toplam = 0`

GÖREV TAHSİSİ

Kod İzleme: `sozlesme_agi(gorevler, maliyet)`



maliyet[r][g] (dk) — yalnız robot bilir

atama

G1 → ?

G2 → ?

G3 → ?

toplam = 0 dk

Duyuru: G1. Boştaki robotlar teklif verir: R1 → 2, R2 → 3, R3 → 6.

Adım 2 / 11

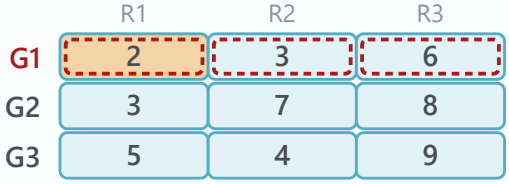
```
1  def sozlesme_agi(gorevler, maliyet):
2      bos = set(maliyet)
3      atama, toplam = {}, 0
▶      for g in gorevler:
▶          teklif = {r: maliyet[r][g] for r in bos}
6          kazanan = min(teklif, key=teklif.get)
7          atama[g] = kazanan
8          toplam += teklif[kazanan]
9          bos.remove(kazanan)
10     return atama, toplam
```

DEĞİŞKENLER

`g = G1` `bos = {R1, R2, R3}` `teklif = {R1: 2, R2: 3, R3: 6}` `toplam = 0`

GÖREV TAHSİSİ

Kod İzleme: `sozlesme_agi(gorevler, maliyet)`



maliyet[r][g] (dk) — yalnız robot bilir

atama

G1 → ?

G2 → ?

G3 → ?

toplam = 0 dk

Ödül: en düşük teklif **R1** (2 dk).

Adım 3 / 11

```
1  def sozlesme_agi(gorevler, maliyet):
2    bos = set(maliyet)
3    atama, toplam = {}, 0
4    for g in gorevler:
5      teklif = {r: maliyet[r][g] for r in bos}
▶      kazanan = min(teklif, key=teklif.get)
7      atama[g] = kazanan
8      toplam += teklif[kazanan]
9      bos.remove(kazanan)
10   return atama, toplam
```

DEĞİŞKENLER

`g = G1`

`teklif = {R1: 2, R2: 3, R3: 6}`

`kazanan = R1` `toplam = 0`

GÖREV TAHSİSİ

Kod İzleme: `sozlesme_agi(gorevler, maliyet)`

	R1 (meşgul)	R2	R3
G1	2	3	6
G2	3	7	8
G3	5	4	9

maliyet[r][g] (dk) — yalnız robot bilir

atama

G1 → R1 (2 dk)

G2 → ?

G3 → ?

toplam = 2 dk

atama[G1] = R1, toplam = 2; R1 artık meşgul.

Adım 4 / 11

```
1  def sozlesme_agi(gorevler, maliyet):
2    bos = set(maliyet)
3    atama, toplam = {}, 0
4    for g in gorevler:
5      teklif = {r: maliyet[r][g] for r in bos}
6      kazanan = min(teklif, key=teklif.get)
▶      atama[g] = kazanan
▶      toplam += teklif[kazanan]
▶      bos.remove(kazanan)
10   return atama, toplam
```

DEĞİŞKENLER

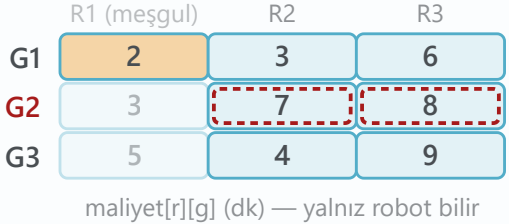
`g = G1` `kazanan = R1`

`atama = {G1: R1}` `toplam = 2`

`bos = {R2, R3}`

GÖREV TAHSİSİ

Kod İzleme: `sozlesme_agi(gorevler, maliyet)`



maliyet[r][g] (dk) — yalnız robot bilir

atama

G1 → R1 (2 dk)

G2 → ?

G3 → ?

toplam = 2 dk

Duyuru: G2. Boştaki robotlar teklif verir: R2 → 7, R3 → 8.

Adım 5 / 11

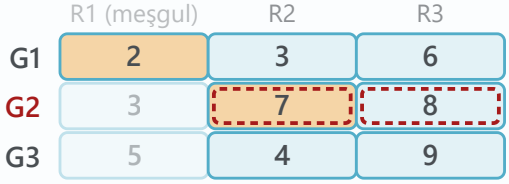
```
1  def sozlesme_agi(gorevler, maliyet):
2      bos = set(maliyet)
3      atama, toplam = {}, 0
▶      for g in gorevler:
▶          teklif = {r: maliyet[r][g] for r in bos}
6          kazanan = min(teklif, key=teklif.get)
7          atama[g] = kazanan
8          toplam += teklif[kazanan]
9          bos.remove(kazanan)
10     return atama, toplam
```

DEĞİŞKENLER

`g = G2` `bos = {R2, R3}` `teklif = {R2: 7, R3: 8}` `toplam = 2`

GÖREV TAHSİSİ

Kod İzleme: `sozlesme_agi(gorevler, maliyet)`



maliyet[r][g] (dk) — yalnız robot bilir

atama

G1 → R1 (2 dk)

G2 → ?

G3 → ?

toplam = 2 dk

Ödül: en düşük teklif **R2** (7 dk).

Adım 6 / 11

```
1  def sozlesme_agi(gorevler, maliyet):
2    bos = set(maliyet)
3    atama, toplam = {}, 0
4    for g in gorevler:
5      teklif = {r: maliyet[r][g] for r in bos}
▶      kazanan = min(teklif, key=teklif.get)
7      atama[g] = kazanan
8      toplam += teklif[kazanan]
9      bos.remove(kazanan)
10   return atama, toplam
```

DEĞİŞKENLER

`g = G2`

`teklif = {R2: 7, R3: 8}`

`kazanan = R2` `toplam = 2`

GÖREV TAHSİSİ

Kod İzleme: `sozlesme_agi(gorevler, maliyet)`

	R1 (meşgul)	R2 (meşgul)	R3
G1	2	3	6
G2	3	7	8
G3	5	4	9

maliyet[r][g] (dk) — yalnız robot bilir

atama

G1 → R1 (2 dk)

G2 → R2 (7 dk)

G3 → ?

toplam = 9 dk

atama[G2] = R2, toplam = 9; R2 artık meşgul. G1'i ucuza alan R1, G2 için de en ucuzdu (3) ama artık yok.

Adım 7 / 11

```
1  def sozlesme_agi(gorevler, maliyet):
2    bos = set(maliyet)
3    atama, toplam = {}, 0
4    for g in gorevler:
5      teklif = {r: maliyet[r][g] for r in bos}
6      kazanan = min(teklif, key=teklif.get)
▶      atama[g] = kazanan
▶      toplam += teklif[kazanan]
▶      bos.remove(kazanan)
10   return atama, toplam
```

DEĞİŞKENLER

`g = G2` `kazanan = R2`

`atama = {G1: R1, G2: R2}`

`toplam = 9` `bos = {R3}`

GÖREV TAHSİSİ

Kod İzleme: `sozlesme_agi(gorevler, maliyet)`



atama

G1 → R1 (2 dk)

G2 → R2 (7 dk)

G3 → ?

toplam = 9 dk

Duyuru: G3. Boştaki robotlar teklif verir: R3 → 9.

Adım 8 / 11

```
1  def sozlesme_agi(gorevler, maliyet):
2    bos = set(maliyet)
3    atama, toplam = {}, 0
▶    for g in gorevler:
▶      teklif = {r: maliyet[r][g] for r in bos}
6      kazanan = min(teklif, key=teklif.get)
7      atama[g] = kazanan
8      toplam += teklif[kazanan]
9      bos.remove(kazanan)
10   return atama, toplam
```

DEĞİŞKENLER

`g = G3` `bos = {R3}`

`teklif = {R3: 9}` `toplam = 9`

GÖREV TAHSİSİ

Kod İzleme: `sozlesme_agi(gorevler, maliyet)`



atama

G1 → R1 (2 dk)

G2 → R2 (7 dk)

G3 → ?

toplam = 9 dk

Ödül: en düşük teklif **R3** (9 dk). Tek teklif: seçim zorunlu.

Adım 9 / 11

```
1  def sozlesme_agi(gorevler, maliyet):
2    bos = set(maliyet)
3    atama, toplam = {}, 0
4    for g in gorevler:
5      teklif = {r: maliyet[r][g] for r in bos}
▶      kazanan = min(teklif, key=teklif.get)
7      atama[g] = kazanan
8      toplam += teklif[kazanan]
9      bos.remove(kazanan)
10   return atama, toplam
```

DEĞİŞKENLER

`g = G3` `teklif = {R3: 9}`

`kazanan = R3` `toplam = 9`

GÖREV TAHSİSİ

Kod İzleme: `sozlesme_agi(gorevler, maliyet)`



maliyet[r][g] (dk) — yalnız robot bilir

atama

G1 → R1 (2 dk)

G2 → R2 (7 dk)

G3 → R3 (9 dk)

toplam = 18 dk

atama[G3] = R3, toplam = 18; R3 artık meşgul.

Adım 10 / 11

```
1  def sozlesme_agi(gorevler, maliyet):
2    bos = set(maliyet)
3    atama, toplam = {}, 0
4    for g in gorevler:
5      teklif = {r: maliyet[r][g] for r in bos}
6      kazanan = min(teklif, key=teklif.get)
▶      atama[g] = kazanan
▶      toplam += teklif[kazanan]
▶      bos.remove(kazanan)
10   return atama, toplam
```

DEĞİŞKENLER

`g = G3` `kazanan = R3`

`atama = {G1: R1, G2: R2, G3: R3}`

`toplam = 18` `bos = set()`

GÖREV TAHSİSİ

Kod İzleme: `sozlesme_agi(gorevler, maliyet)`

	R1	R2	R3
G1	2	3	6
G2	3	7	8
G3	5	4	9

maliyet[r][g] (dk) — yalnız robot bilir

atama

G1 → R1 (2 dk)

G2 → R2 (7 dk)

G3 → R3 (9 dk)

toplam = 18 dk

Sonuç 18 dk: her adımda yerel en iyi karar, ama toplam optimum değil.

Adım 11 / 11

```
1 def sozlesme_agi(gorevler, maliyet):
2   bos = set(maliyet)
3   atama, toplam = {}, 0
4   for g in gorevler:
5     teklif = {r: maliyet[r][g] for r in bos}
6     kazanan = min(teklif, key=teklif.get)
7     atama[g] = kazanan
8     toplam += teklif[kazanan]
9     bos.remove(kazanan)
▶   return atama, toplam
```

DEĞİŞKENLER

`atama = {G1: R1, G2: R2, G3: R3}`

`toplam = 18`

GÖREV TAHSİSİ

Kod İzleme: `optimum(gorevler, maliyet)`

	R1	R2	R3
G1	2	3	6
G2	3	7	8
G3	5	4	9

(R1, R2, R3) · (R2, R3, R1) ·
(R1, R3, R2) · (R3, R1, R2) ·
(R2, R1, R3) · (R3, R2, R1) ·

p = (G1, G2, G3) görevlerini alan robotlar; turuncu = en iyi

Merkezi çözüm: tüm maliyetleri bilen yönetici, 3 robotun 3! = 6 sıralamasını dener.

Adım 1 / 8

```
1 def optimum(gorevler, maliyet):
▶   en_iyi = None
3   for p in permutations(maliyet):
4     t = sum(maliyet[r][g]
5             for g, r in zip(gorevler, p))
6     if en_iyi is None or t < en_iyi[0]:
7       en_iyi = (t, p)
8   return en_iyi
```

DEĞİŞKENLER

`en_iyi = None`

GÖREV TAHSİSİ

Kod İzleme: `optimum(gorevler, maliyet)`

	R1	R2	R3
G1	2	3	6
G2	3	7	8
G3	5	4	9

(R1, R2, R3) 18 (R2, R3, R1) ·
(R1, R3, R2) · (R3, R1, R2) ·
(R2, R1, R3) · (R3, R2, R1) ·

p = (G1, G2, G3) görevlerini alan robotlar; turuncu = en iyi

p = (R1, R2, R3): t = 2 + 7 + 9 = **18** ⇒ yeni en iyi.

Adım 2 / 8

```
1 def optimum(gorevler, maliyet):
2   en_iyi = None
▶   for p in permutations(maliyet):
▶     t = sum(maliyet[r][g]
▶             for g, r in zip(gorevler, p))
▶     if en_iyi is None or t < en_iyi[0]:
▶       en_iyi = (t, p)
8   return en_iyi
```

DEĞİŞKENLER

`p = (R1, R2, R3)` `t = 18`

`en_iyi = (18, (R1, R2, R3))`

Kod İzleme: `optimum(gorevler, maliyet)`

	R1	R2	R3
G1	2	3	6
G2	3	7	8
G3	5	4	9

(R1, R2, R3) 18 (R2, R3, R1) ·
(R1, R3, R2) 14 (R3, R1, R2) ·
(R2, R1, R3) · (R3, R2, R1) ·

p = (G1, G2, G3) görevlerini alan robotlar; turuncu = en iyi

p = (R1, R3, R2): t = 2 + 8 + 4 = **14** ⇒ yeni en iyi.

Adım 3 / 8

```
1 def optimum(gorevler, maliyet):
2   en_iyi = None
▶   for p in permutations(maliyet):
▶     t = sum(maliyet[r][g]
▶             for g, r in zip(gorevler, p))
▶     if en_iyi is None or t < en_iyi[0]:
▶       en_iyi = (t, p)
8   return en_iyi
```

DEĞİŞKENLER

`p = (R1, R3, R2)` `t = 14`

`en_iyi = (14, (R1, R3, R2))`

Kod İzleme: `optimum(gorevler, maliyet)`

	R1	R2	R3
G1	2	3	6
G2	3	7	8
G3	5	4	9

(R1, R2, R3) 18 (R2, R3, R1) ·

(R1, R3, R2) 14 (R3, R1, R2) ·

(R2, R1, R3) 15 (R3, R2, R1) ·

p = (G1, G2, G3) görevlerini alan robotlar; turuncu = en iyi

p = (R2, R1, R3): t = 3 + 3 + 9 = 15 ≥ 14: değişmez.

Adım 4 / 8

```
1 def optimum(gorevler, maliyet):
2   en_iyi = None
▶   for p in permutations(maliyet):
▶     t = sum(maliyet[r][g]
▶             for g, r in zip(gorevler, p))
▶     if en_iyi is None or t < en_iyi[0]:
7       en_iyi = (t, p)
8   return en_iyi
```

DEĞİŞKENLER

`p = (R2, R1, R3)` `t = 15`

`en_iyi = (14, (R1, R3, R2))`

Kod İzleme: `optimum(gorevler, maliyet)`

	R1	R2	R3
G1	2	3	6
G2	3	7	8
G3	5	4	9

(R1, R2, R3) 18
(R1, R3, R2) 14
(R2, R1, R3) 15
(R2, R3, R1) 16
(R3, R1, R2) ·
(R3, R2, R1) ·

p = (G1, G2, G3) görevlerini alan robotlar; turuncu = en iyi

p = (R2, R3, R1): t = 3 + 8 + 5 = 16 ≥ 14: değişmez.

Adım 5 / 8

```
1 def optimum(gorevler, maliyet):
2   en_iyi = None
▶   for p in permutations(maliyet):
▶     t = sum(maliyet[r][g]
▶             for g, r in zip(gorevler, p))
▶     if en_iyi is None or t < en_iyi[0]:
7       en_iyi = (t, p)
8   return en_iyi
```

DEĞİŞKENLER

`p = (R2, R3, R1)` `t = 16`

`en_iyi = (14, (R1, R3, R2))`

Kod İzleme: `optimum(gorevler, maliyet)`

	R1	R2	R3
G1	2	3	6
G2	3	7	8
G3	5	4	9

(R1, R2, R3) 18
(R1, R3, R2) 14
(R2, R1, R3) 15
(R2, R3, R1) 16
(R3, R1, R2) 13
(R3, R2, R1) ·

p = (G1, G2, G3) görevlerini alan robotlar; turuncu = en iyi

p = (R3, R1, R2): t = 6 + 3 + 4 = **13** ⇒ yeni en iyi.


```
1 def optimum(gorevler, maliyet):
2   en_iyi = None
▶   for p in permutations(maliyet):
▶     t = sum(maliyet[r][g]
▶             for g, r in zip(gorevler, p))
▶     if en_iyi is None or t < en_iyi[0]:
▶       en_iyi = (t, p)
8   return en_iyi
```

DEĞİŞKENLER

`p = (R3, R1, R2)` `t = 13`

`en_iyi = (13, (R3, R1, R2))`

Kod İzleme: `optimum(gorevler, maliyet)`

	R1	R2	R3
G1	2	3	6
G2	3	7	8
G3	5	4	9

(R1, R2, R3)	18	(R2, R3, R1)	16
(R1, R3, R2)	14	(R3, R1, R2)	13
(R2, R1, R3)	15	(R3, R2, R1)	18

p = (G1, G2, G3) görevlerini alan robotlar; turuncu = en iyi

p = (R3, R2, R1): t = 6 + 7 + 5 = 18 ≥ 13: değişmez.

Adım 7 / 8

```
1 def optimum(gorevler, maliyet):
2   en_iyi = None
▶   for p in permutations(maliyet):
▶     t = sum(maliyet[r][g]
▶             for g, r in zip(gorevler, p))
▶     if en_iyi is None or t < en_iyi[0]:
7       en_iyi = (t, p)
8   return en_iyi
```

DEĞİŞKENLER

`p = (R3, R2, R1)` `t = 18`

`en_iyi = (13, (R3, R1, R2))`

GÖREV TAHSİSİ

Kod İzleme: `optimum(gorevler, maliyet)`

	R1	R2	R3
G1	2	3	6
G2	3	7	8
G3	5	4	9

(R1, R2, R3) 18 (R2, R3, R1) 16
(R1, R3, R2) 14 (R3, R1, R2) 13
(R2, R1, R3) 15 (R3, R2, R1) 18

p = (G1, G2, G3) görevlerini alan robotlar; turuncu = en iyi

Optimum 13 dk: G1→R3, G2→R1, G3→R2; sözleşme ağı 18 dk (%38 fazla). Ama n robot için n! atama: n = 10'da 3 628 800 (Macar algoritması $O(n^3)$ ile çözer).

Adım 8 / 8

```
1 def optimum(gorevler, maliyet):
2   en_iyi = None
3   for p in permutations(maliyet):
4     t = sum(maliyet[r][g]
5             for g, r in zip(gorevler, p))
6     if en_iyi is None or t < en_iyi[0]:
7       en_iyi = (t, p)
▶   return en_iyi
```

DEĞİŞKENLER

`en_iyi = (13, (R3, R1, R2))`

GÖREV TAHSİSİ

Sözleşme Ağı: Güçlü ve Zayıf Yanlar

Güçlü yanlar

+ Dağıtık: yönetici maliyet tablosunu bilmeden iyi bir atama yapar.
+ Görev başına mesaj sayısı O(n): duyuru + n teklif + ödül.
+ Yeni robot eklemek kolay; bozulan robotun görevi yeniden ihaleye çıkar.

Zayıf yanlar

− **Açgözlü ve sıralı**: örnekte 18 dk, optimum 13 dk (%38 fazla). G1'i ucuza alan R1, G2 için en iyi aday olduğu hâlde meşgul kaldı.
− Robotlar maliyetini stratejik olarak şişirebilir.
− Görev sırası sonucu değiştirir.

İyileştirmeler: **yeniden ihale** (atamaları takasla düzeltme), **kombinatoryal açık artırma** (robotlar görev paketlerine teklif verir), dürüst teklif için **ikinci fiyat / VCG ödemeleri**.

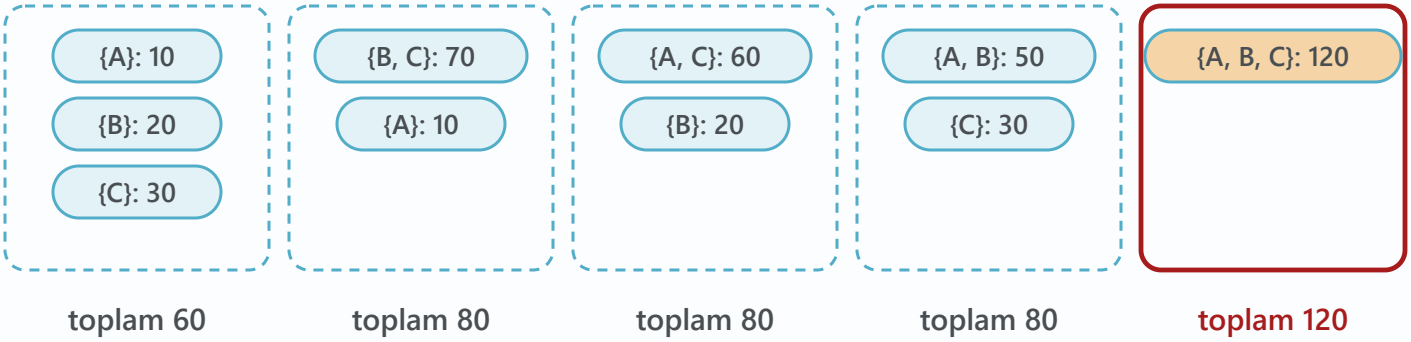
Koalisyon Oluşumu (Coalition Formation)

- Tek başına hedefe ulaşamayan ajanlar geçici veya kalıcı **koalisyonlar** kurabilir: kargo firmaları araç paylaşır, drone'lar ağır yükü birlikte taşır.
- Bu oyunlar **işbirlikçidir**: bağlayıcı anlaşma yapılabilir; soru "hangi strateji?" değil, "kim kiminle birleşir ve kazanç **nasıl paylaşılır**?"
- Paylaşım adil değilse koalisyon dağılır: kararlılık doğrudan paylaşım kuralına bağlıdır.

Shapley değeri
Adalet: herkes ortalama *marjinal katkısı* kadar alır.

Çekirdek (core)
Kararlılık: hiçbir alt grup ayrılarak daha fazlasını alamaz.

Koalisyon Yapıları: Kim Kiminle?



- **Koalisyon yapısı**: ajanların ayrık koalisyonlara bölünmesi. Toplum için soru: toplam değeri en büyük yapı hangisi?
- Yapı sayısı Bell sayısıdır: n = 3 için 5 n = 4 için 15 n = 8 için 4 140 n = 10 için 115 975.
- Örnekte oyun **süperadditif**: birleşmek hiç zarar vermez, en iyi yapı büyük koalisyon {A, B, C} (120).
- Genel durumda en iyi yapıyı bulmak NP-zordur (Sandholm vd., 1999); geriye kalan soru **120'nin paylaşımı**.

Koalisyon Oyunu ve Karakteristik Fonksiyon

- Oyuncular kümesi N = {1, …, n}; her **koalisyon** S ⊆ N için **v(S)**: S'nin üyeleri birlikte, diğerlerinden bağımsız olarak elde edebileceği toplam değer.
- v: $2^N \to \mathbb{R}$, v(∅) = 0. Buna **karakteristik fonksiyon** denir; (N, v) aktarılabilir faydalı (TU) oyundur.
- **Süperadditif**: ayrık S, T için v(S ∪ T) ≥ v(S) + v(T) → birleşmek zarar vermez; büyük koalisyon N oluşur.
- Örnekte üç lojistik firması A, B, C: birlikte 120, en iyi ikili 70, tek başına en fazla 30 kazanır.

S	v(S)
{A}	10
{B}	20
{C}	30
{A, B}	50
{A, C}	60
{B, C}	70
{A, B, C}	120

v(S) — kâr (bin TL)

120 Nasıl Paylaşılmalı?

Bir **paylaşım** (imputation) $x = (x_A, x_B, x_C)$: toplamı $v(N) = 120$ (verimlilik) ve herkes en az tek başına değerini alır (bireysel rasyonellik).

Öneri	x_A	x_B	x_C	Gerekçe / sorun
Eşit bölüşüm	40	40	40	Basit; ama C'nin daha fazla katkısını yok sayar
Tek başına değerle orantılı (10 : 20 : 30)	20	40	60	Yalnızca $v(\{i\})$'ye bakar; sinerjileri görmez
Güçlü C'nin önerisi	10	25	85	A ve B birlikte 50 alabilir; kabul etmezler
Shapley değeri	30	40	50	Ortalama marjinal katkı

İki ölçüt: paylaşım **adil** mi (Shapley) ve **kararlı** mı (çekirdek)?

KOALİSYON OYUNLARI

Shapley Değeri (Shapley Value)

- Oyuncular **rastgele bir sırayla** gelip koalisyona katılır. Her oyuncuya, geldiği anda kattığı değer (marjinal katkı) verilir.
- Shapley değeri: n! sıranın hepsi üzerinden **ortalama marjinal katkı**.
- Hesap maliyeti: n! sıra (ya da 2^{n-1} alt küme); büyük n için örnekleme ile yaklaşık hesaplanır.
- Shapley (1953): yandaki dört aksiyomu sağlayan **tek** kural. Model açıklamada (SHAP) da kullanılır.

Verimlilik

$\Sigma\ \varphi_i = v(N)$.

Simetri

Aynı katkıyı yapan iki oyuncu aynı payı alır.

Kukla oyuncu

Hiçbir koalisyona katkısı yoksa $\varphi_i = 0$.

Toplamsallık

$\varphi(v + w) = \varphi(v) + \varphi(w)$.

Eşdeğer formül: $\varphi_i = \Sigma_{S \subseteq N \setminus \{i\}} |S|!\ (n - |S| - 1)!\ /\ n! \cdot [\ v(S \cup \{i\}) - v(S)\ ]$

Shapley Değeri Adım Adım: 6 Sıralama

S	v(S)
{A}	10
{B}	20
{C}	30
{A, B}	50
{A, C}	60
{B, C}	70
{A, B, C}	120

v(S)

sıra	A	B	C
A → B → C	·	·	·
A → C → B	·	·	·
B → A → C	·	·	·
B → C → A	·	·	·
C → A → B	·	·	·
C → B → A	·	·	·
toplam	·	·	·
φ = toplam / 6	·	·	·

Her sırada ajanlar tek tek gelir; her birine **geldiği anda koalisyona kattığı değer** (marjinal katkı) yazılır.

Shapley Değeri Adım Adım: 6 Sıralama

S	v(S)
{A}	10
{B}	20
{C}	30
{A, B}	50
{A, C}	60
{B, C}	70
{A, B, C}	120

v(S)

sıra	A	B	C
A → B → C	10	40	70
A → C → B	·	·	·
B → A → C	·	·	·
B → C → A	·	·	·
C → A → B	·	·	·
C → B → A	·	·	·
toplam	·	·	·
φ = toplam / 6	·	·	·

Sıra A → B → C — A: 10 − 0 = 10; B: 50 − 10 = 40; C: 120 − 50 = 70. Satır toplamı 120 = v(ABC).

Shapley Değeri Adım Adım: 6 Sıralama

S	v(S)
{A}	10
{B}	20
{C}	30
{A, B}	50
{A, C}	60
{B, C}	70
{A, B, C}	120

v(S)

sıra	A	B	C
A → B → C	10	40	70
A → C → B	10	60	50
B → A → C	·	·	·
B → C → A	·	·	·
C → A → B	·	·	·
C → B → A	·	·	·
toplam	·	·	·
φ = toplam / 6	·	·	·

Sıra A → C → B — A: 10 − 0 = 10; C: 60 − 10 = 50; B: 120 − 60 = 60. Satır toplamı 120 = v(ABC).

Shapley Değeri Adım Adım: 6 Sıralama

S	v(S)
{A}	10
{B}	20
{C}	30
{A, B}	50
{A, C}	60
{B, C}	70
{A, B, C}	120

v(S)

sıra	A	B	C
A → B → C	10	40	70
A → C → B	10	60	50
B → A → C	30	20	70
B → C → A	·	·	·
C → A → B	·	·	·
C → B → A	·	·	·
toplam	·	·	·
φ = toplam / 6	·	·	·

Sıra B → A → C — B: 20 − 0 = 20; A: 50 − 20 = 30; C: 120 − 50 = 70. Satır toplamı 120 = v(ABC).

Shapley Değeri Adım Adım: 6 Sıralama

S	v(S)
{A}	10
{B}	20
{C}	30
{A, B}	50
{A, C}	60
{B, C}	70
{A, B, C}	120

v(S)

sıra	A	B	C
A → B → C	10	40	70
A → C → B	10	60	50
B → A → C	30	20	70
B → C → A	50	20	50
C → A → B	·	·	·
C → B → A	·	·	·
toplam	·	·	·
φ = toplam / 6	·	·	·

Sıra B → C → A — B: 20 − 0 = 20; C: 70 − 20 = 50; A: 120 − 70 = 50. Satır toplamı 120 = v(ABC).

Shapley Değeri Adım Adım: 6 Sıralama

S	v(S)
{A}	10
{B}	20
{C}	30
{A, B}	50
{A, C}	60
{B, C}	70
{A, B, C}	120

v(S)

sıra	A	B	C
A → B → C	10	40	70
A → C → B	10	60	50
B → A → C	30	20	70
B → C → A	50	20	50
C → A → B	30	60	30
C → B → A	·	·	·
toplam	·	·	·
φ = toplam / 6	·	·	·

Sıra C → A → B — C: 30 − 0 = 30; A: 60 − 30 = 30; B: 120 − 60 = 60. Satır toplamı 120 = v(ABC).

Shapley Değeri Adım Adım: 6 Sıralama

S	v(S)
{A}	10
{B}	20
{C}	30
{A, B}	50
{A, C}	60
{B, C}	70
{A, B, C}	120

v(S)

sıra	A	B	C
A → B → C	10	40	70
A → C → B	10	60	50
B → A → C	30	20	70
B → C → A	50	20	50
C → A → B	30	60	30
C → B → A	50	40	30
toplam	·	·	·
φ = toplam / 6	·	·	·

Sıra C → B → A — C: 30 − 0 = 30; B: 70 − 30 = 40; A: 120 − 70 = 50. Satır toplamı 120 = v(ABC).

Shapley Değeri Adım Adım: 6 Sıralama

S	v(S)
{A}	10
{B}	20
{C}	30
{A, B}	50
{A, C}	60
{B, C}	70
{A, B, C}	120

v(S)

sıra	A	B	C
A → B → C	10	40	70
A → C → B	10	60	50
B → A → C	30	20	70
B → C → A	50	20	50
C → A → B	30	60	30
C → B → A	50	40	30
toplam	180	240	300
φ = toplam / 6	**30**	**40**	**50**

Ortalama marjinal katkı: **φ = (30, 40, 50)**. Toplam 120: verimlilik aksiyomu sağlanır.

Kod İzleme: `shapley(N, v)` (1/2)

sıra (katılım sırası)

S = ∅

φA 0 | φB 0 | φC 0

phi (toplam katkı, henüz bölünmedi)

phi sözlüğü sıfırlanır; `permutations` (itertools) 3! = 6 sırayı üretir.

Adım 1 / 13

```
1  def shapley(N, v):
▶    phi = {i: 0 for i in N}
▶    siralar = list(permutations(N))
4    for sira in siralar:
5      S = frozenset()
6      for i in sira:
7        katki = v[S | {i}] - v[S]
8        phi[i] += katki
9        S = S | {i}
10   return {i: phi[i] / len(siralar) for i in N}
```

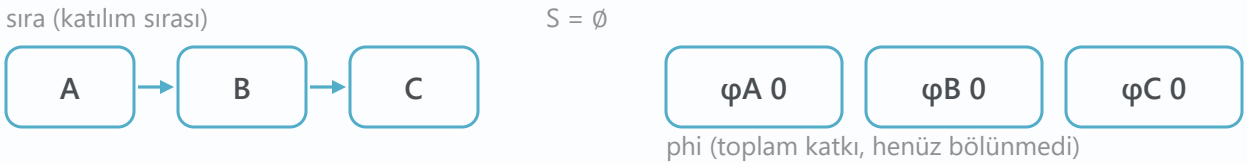
DEĞİŞKENLER

`N = [A, B, C]`

`siralar = 6 sıra` | `phi[A] = 0`

`phi[B] = 0` | `phi[C] = 0`

Kod İzleme: `shapley(N, v)` (1/2)



1. sıra (A, B, C): ajanlar bu sırayla gelir. S boş koalisyonla başlar.

Adım 2 / 13

```
1  def shapley(N, v):
2    phi = {i: 0 for i in N}
3    siralar = list(permutations(N))
▶    for sira in siralar:
▶      S = frozenset()
6      for i in sira:
7        katki = v[S | {i}] - v[S]
8        phi[i] += katki
9        S = S | {i}
10   return {i: phi[i] / len(siralar) for i in N}
```

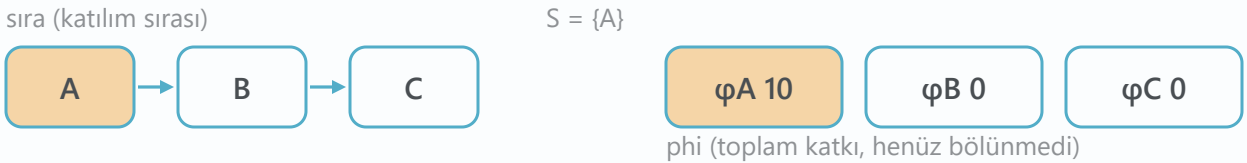
DEĞİŞKENLER

`sira = (A, B, C)` `S = ∅`

`phi[A] = 0` `phi[B] = 0`

`phi[C] = 0`

Kod İzleme: `shapley(N, v)` (1/2)



A katılır: katkı = v(A) − v(∅) = 10 − 0 = **10**; phi[A] = 10.

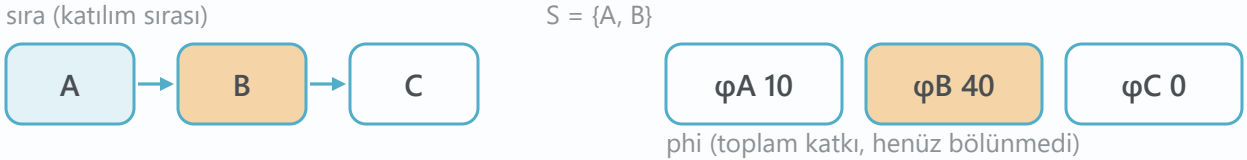
Adım 3 / 13

```
1  def shapley(N, v):
2    phi = {i: 0 for i in N}
3    siralar = list(permutations(N))
4    for sira in siralar:
5      S = frozenset()
▶      for i in sira:
▶        katki = v[S | {i}] - v[S]
▶        phi[i] += katki
▶        S = S | {i}
10   return {i: phi[i] / len(siralar) for i in N}
```

DEĞİŞKENLER

`sira = (A, B, C)` `S = {A}` `i = A` `katki = 10` `phi[A] = 10` `phi[B] = 0` `phi[C] = 0`

Kod İzleme: `shapley(N, v)` (1/2)



B katılır: katkı = v(AB) − v(A) = 50 − 10 = **40**; phi[B] = 40.

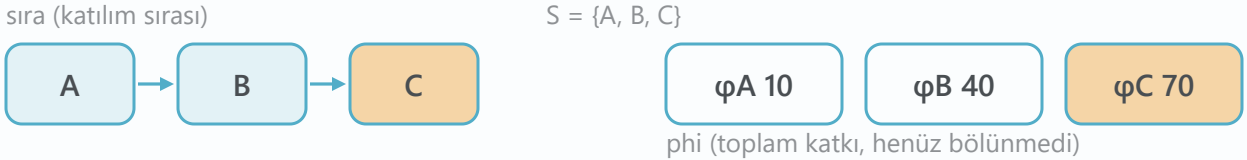
Adım 4 / 13

```
1  def shapley(N, v):
2    phi = {i: 0 for i in N}
3    siralar = list(permutations(N))
4    for sira in siralar:
5      S = frozenset()
▶      for i in sira:
▶        katki = v[S | {i}] - v[S]
▶        phi[i] += katki
▶        S = S | {i}
10   return {i: phi[i] / len(siralar) for i in N}
```

DEĞİŞKENLER

`sira = (A, B, C)` `S = {A, B}` `i = B` `katki = 40` `phi[A] = 10` `phi[B] = 40` `phi[C] = 0`

Kod İzleme: `shapley(N, v)` (1/2)



C katılır: katkı = v(ABC) − v(AB) = 120 − 50 = **70**; phi[C] = 70.

Adım 5 / 13

```
1  def shapley(N, v):
2    phi = {i: 0 for i in N}
3    siralar = list(permutations(N))
4    for sira in siralar:
5      S = frozenset()
▶      for i in sira:
▶        katki = v[S | {i}] - v[S]
▶        phi[i] += katki
▶        S = S | {i}
10   return {i: phi[i] / len(siralar) for i in N}
```

DEĞİŞKENLER

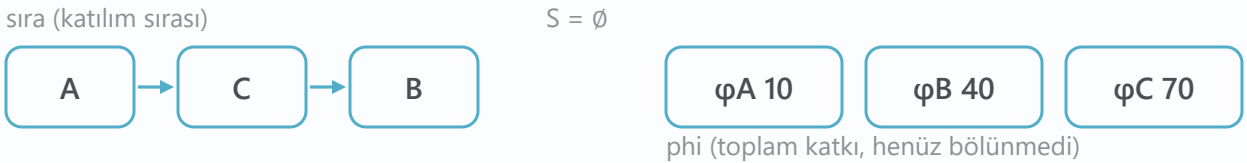
`sira = (A, B, C)`

`S = {A, B, C}` `i = C`

`katki = 70` `phi[A] = 10`

`phi[B] = 40` `phi[C] = 70`

Kod İzleme: `shapley(N, v)` (1/2)



2. sıra (A, C, B): ajanlar bu sırayla gelir. S boş koalisyonla başlar.

Adım 6 / 13

```
1  def shapley(N, v):
2    phi = {i: 0 for i in N}
3    siralar = list(permutations(N))
▶    for sira in siralar:
▶      S = frozenset()
6      for i in sira:
7        katki = v[S | {i}] - v[S]
8        phi[i] += katki
9        S = S | {i}
10   return {i: phi[i] / len(siralar) for i in N}
```

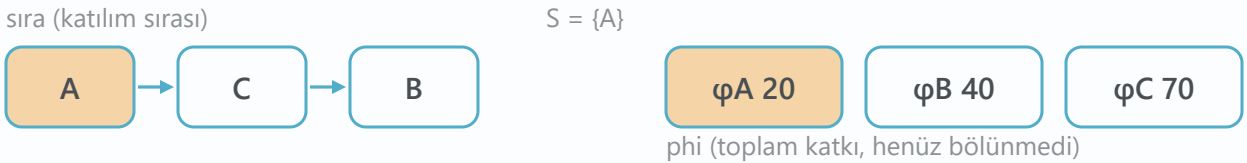
DEĞİŞKENLER

`sira = (A, C, B)` `S = ∅`

`phi[A] = 10` `phi[B] = 40`

`phi[C] = 70`

Kod İzleme: `shapley(N, v)` (1/2)



A katılır: katkı = v(A) − v(∅) = 10 − 0 = **10**; phi[A] = 20.

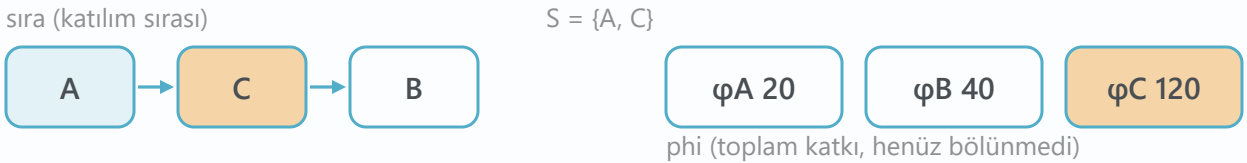
Adım 7 / 13

```
1  def shapley(N, v):
2    phi = {i: 0 for i in N}
3    siralar = list(permutations(N))
4    for sira in siralar:
5      S = frozenset()
▶      for i in sira:
▶        katki = v[S | {i}] - v[S]
▶        phi[i] += katki
▶        S = S | {i}
10   return {i: phi[i] / len(siralar) for i in N}
```

DEĞİŞKENLER

`sira = (A, C, B)` `S = {A}` `i = A` `katki = 10` `phi[A] = 20` `phi[B] = 40` `phi[C] = 70`

Kod İzleme: `shapley(N, v)` (1/2)



C katılır: katkı = v(AC) − v(A) = 60 − 10 = **50**; phi[C] = 120.

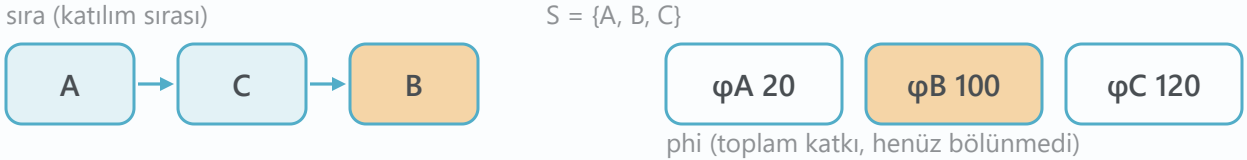
Adım 8 / 13

```
1  def shapley(N, v):
2    phi = {i: 0 for i in N}
3    siralar = list(permutations(N))
4    for sira in siralar:
5      S = frozenset()
▶      for i in sira:
▶        katki = v[S | {i}] - v[S]
▶        phi[i] += katki
▶        S = S | {i}
10   return {i: phi[i] / len(siralar) for i in N}
```

DEĞİŞKENLER

`sira = (A, C, B)` `S = {A, C}` `i = C` `katki = 50` `phi[A] = 20` `phi[B] = 40` `phi[C] = 120`

Kod İzleme: `shapley(N, v)` (1/2)



B katılır: katkı = v(ABC) − v(AC) = 120 − 60 = **60**; phi[B] = 100.

Adım 9 / 13

```
1  def shapley(N, v):
2    phi = {i: 0 for i in N}
3    siralar = list(permutations(N))
4    for sira in siralar:
5      S = frozenset()
▶      for i in sira:
▶        katki = v[S | {i}] - v[S]
▶        phi[i] += katki
▶        S = S | {i}
10   return {i: phi[i] / len(siralar) for i in N}
```

DEĞİŞKENLER

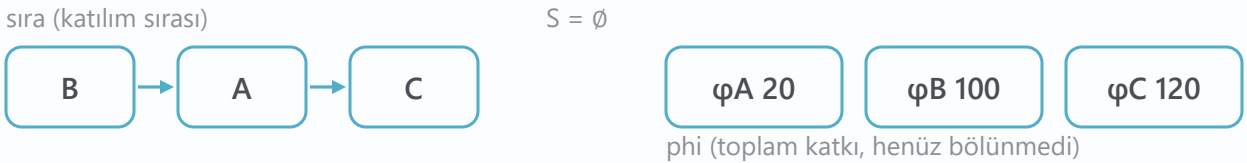
`sira = (A, C, B)`

`S = {A, B, C}` `i = B`

`katki = 60` `phi[A] = 20`

`phi[B] = 100` `phi[C] = 120`

Kod İzleme: `shapley(N, v)` (1/2)



3. sıra (B, A, C): ajanlar bu sırayla gelir. S boş koalisyonla başlar.

Adım 10 / 13

```
1  def shapley(N, v):
2    phi = {i: 0 for i in N}
3    siralar = list(permutations(N))
▶    for sira in siralar:
▶      S = frozenset()
6      for i in sira:
7        katki = v[S | {i}] - v[S]
8        phi[i] += katki
9        S = S | {i}
10   return {i: phi[i] / len(siralar) for i in N}
```

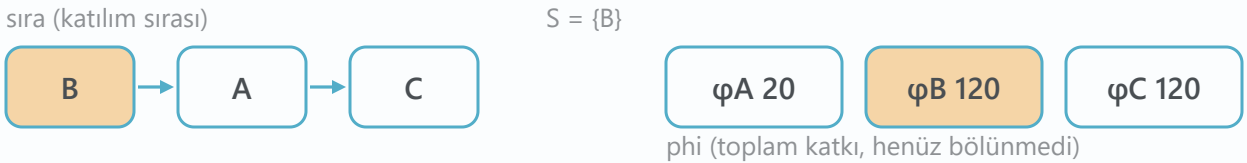
DEĞİŞKENLER

`sira = (B, A, C)` `S = ∅`

`phi[A] = 20` `phi[B] = 100`

`phi[C] = 120`

Kod İzleme: `shapley(N, v)` (1/2)



B katılır: katkı = v(B) − v(∅) = 20 − 0 = **20**; phi[B] = 120.

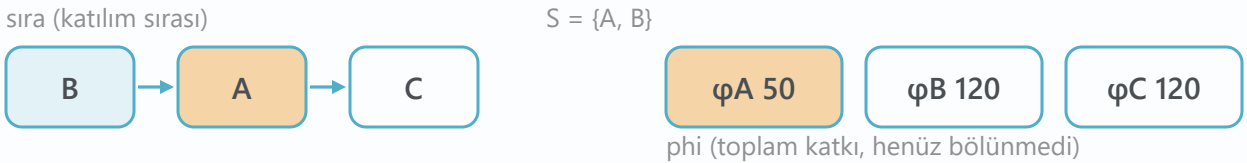
Adım 11 / 13

```
1  def shapley(N, v):
2    phi = {i: 0 for i in N}
3    siralar = list(permutations(N))
4    for sira in siralar:
5      S = frozenset()
▶      for i in sira:
▶        katki = v[S | {i}] - v[S]
▶        phi[i] += katki
▶        S = S | {i}
10   return {i: phi[i] / len(siralar) for i in N}
```

DEĞİŞKENLER

`sira = (B, A, C)` `S = {B}` `i = B` `katki = 20` `phi[A] = 20` `phi[B] = 120` `phi[C] = 120`

Kod İzleme: `shapley(N, v)` (1/2)



A katılır: katkı = v(AB) − v(B) = 50 − 20 = **30**; phi[A] = 50.

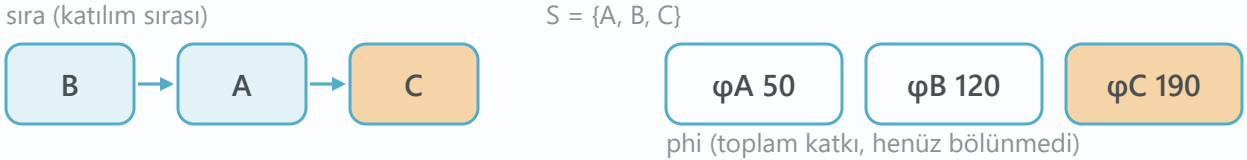
Adım 12 / 13

```
1  def shapley(N, v):
2    phi = {i: 0 for i in N}
3    siralar = list(permutations(N))
4    for sira in siralar:
5      S = frozenset()
▶      for i in sira:
▶        katki = v[S | {i}] - v[S]
▶        phi[i] += katki
▶        S = S | {i}
10   return {i: phi[i] / len(siralar) for i in N}
```

DEĞİŞKENLER

`sira = (B, A, C)` `S = {A, B}` `i = A` `katki = 30` `phi[A] = 50` `phi[B] = 120` `phi[C] = 120`

Kod İzleme: `shapley(N, v)` (1/2)



C katılır: katkı = v(ABC) − v(AB) = 120 − 50 = **70**; phi[C] = 190.

Adım 13 / 13

```
1  def shapley(N, v):
2    phi = {i: 0 for i in N}
3    siralar = list(permutations(N))
4    for sira in siralar:
5      S = frozenset()
▶      for i in sira:
▶        katki = v[S | {i}] - v[S]
▶        phi[i] += katki
▶        S = S | {i}
10   return {i: phi[i] / len(siralar) for i in N}
```

DEĞİŞKENLER

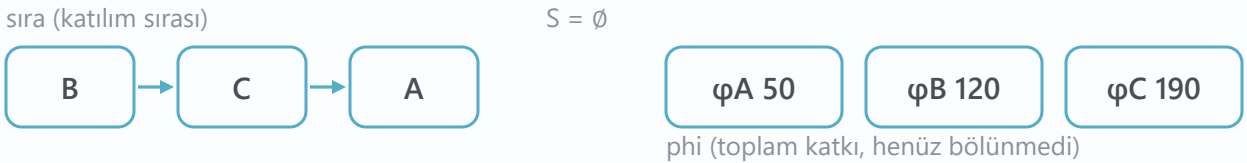
`sira = (B, A, C)`

`S = {A, B, C}` `i = C`

`katki = 70` `phi[A] = 50`

`phi[B] = 120` `phi[C] = 190`

Kod İzleme: `shapley(N, v)` (2/2)



4. sıra (B, C, A): ajanlar bu sırayla gelir. S boş koalisyonla başlar.

Adım 1 / 13

```
1  def shapley(N, v):
2    phi = {i: 0 for i in N}
3    siralar = list(permutations(N))
▶    for sira in siralar:
▶      S = frozenset()
6      for i in sira:
7        katki = v[S | {i}] - v[S]
8        phi[i] += katki
9        S = S | {i}
10   return {i: phi[i] / len(siralar) for i in N}
```

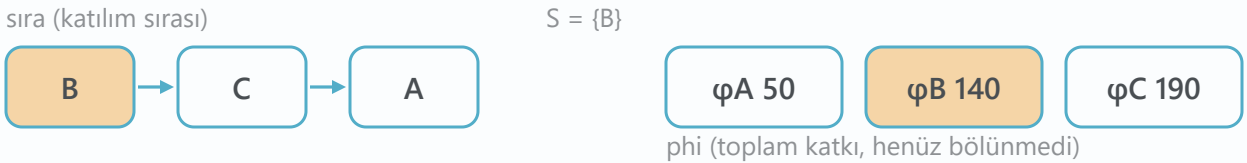
DEĞİŞKENLER

`sira = (B, C, A)` `S = ∅`

`phi[A] = 50` `phi[B] = 120`

`phi[C] = 190`

Kod İzleme: `shapley(N, v)` (2/2)



B katılır: katkı = v(B) − v(∅) = 20 − 0 = **20**; phi[B] = 140.

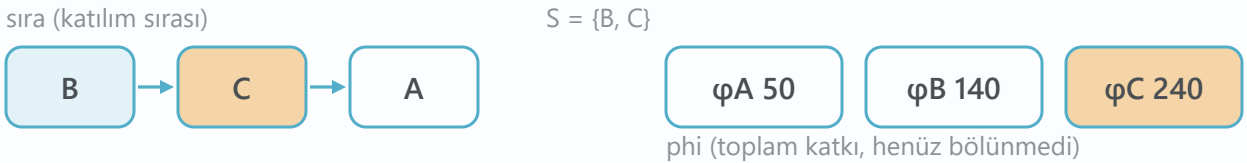
Adım 2 / 13

```
1  def shapley(N, v):
2    phi = {i: 0 for i in N}
3    siralar = list(permutations(N))
4    for sira in siralar:
5      S = frozenset()
▶      for i in sira:
▶        katki = v[S | {i}] - v[S]
▶        phi[i] += katki
▶        S = S | {i}
10   return {i: phi[i] / len(siralar) for i in N}
```

DEĞİŞKENLER

`sira = (B, C, A)` `S = {B}` `i = B` `katki = 20` `phi[A] = 50` `phi[B] = 140` `phi[C] = 190`

Kod İzleme: `shapley(N, v)` (2/2)



C katılır: katkı = v(BC) − v(B) = 70 − 20 = **50**; phi[C] = 240.

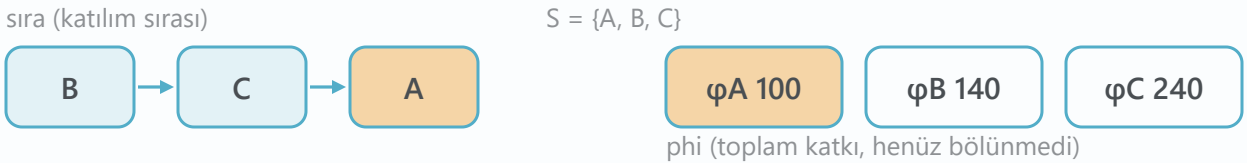
Adım 3 / 13

```
1  def shapley(N, v):
2    phi = {i: 0 for i in N}
3    siralar = list(permutations(N))
4    for sira in siralar:
5      S = frozenset()
▶      for i in sira:
▶        katki = v[S | {i}] - v[S]
▶        phi[i] += katki
▶        S = S | {i}
10   return {i: phi[i] / len(siralar) for i in N}
```

DEĞİŞKENLER

`sira = (B, C, A)` `S = {B, C}` `i = C` `katki = 50` `phi[A] = 50` `phi[B] = 140` `phi[C] = 240`

Kod İzleme: `shapley(N, v)` (2/2)



A katılır: katkı = v(ABC) − v(BC) = 120 − 70 = **50**; phi[A] = 100.

Adım 4 / 13

```
1  def shapley(N, v):
2    phi = {i: 0 for i in N}
3    siralar = list(permutations(N))
4    for sira in siralar:
5      S = frozenset()
▶      for i in sira:
▶        katki = v[S | {i}] - v[S]
▶        phi[i] += katki
▶        S = S | {i}
10   return {i: phi[i] / len(siralar) for i in N}
```

DEĞİŞKENLER

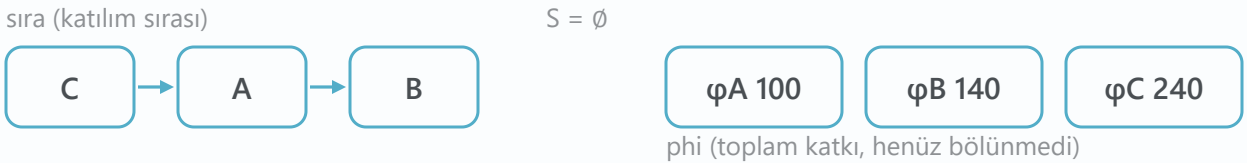
`sira = (B, C, A)`

`S = {A, B, C}` `i = A`

`katki = 50` `phi[A] = 100`

`phi[B] = 140` `phi[C] = 240`

Kod İzleme: `shapley(N, v)` (2/2)



5. sıra (C, A, B): ajanlar bu sırayla gelir. S boş koalisyonla başlar.

Adım 5 / 13

```
1  def shapley(N, v):
2    phi = {i: 0 for i in N}
3    siralar = list(permutations(N))
▶    for sira in siralar:
▶      S = frozenset()
6      for i in sira:
7        katki = v[S | {i}] - v[S]
8        phi[i] += katki
9        S = S | {i}
10   return {i: phi[i] / len(siralar) for i in N}
```

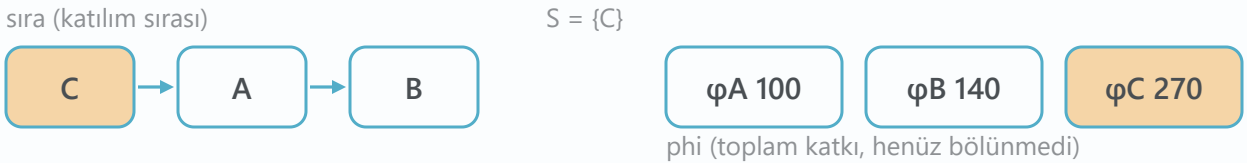
DEĞİŞKENLER

`sira = (C, A, B)` `S = ∅`

`phi[A] = 100` `phi[B] = 140`

`phi[C] = 240`

Kod İzleme: `shapley(N, v)` (2/2)



C katılır: katkı = v(C) − v(∅) = 30 − 0 = **30**; phi[C] = 270.

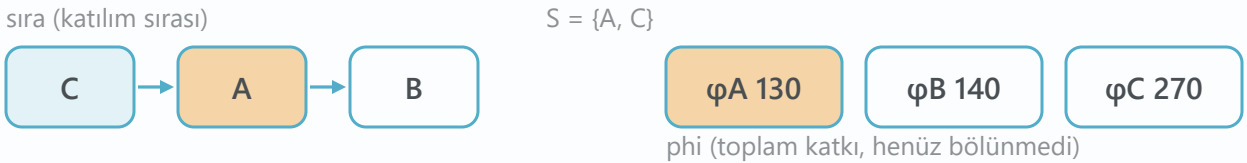
Adım 6 / 13

```
1  def shapley(N, v):
2    phi = {i: 0 for i in N}
3    siralar = list(permutations(N))
4    for sira in siralar:
5      S = frozenset()
▶      for i in sira:
▶        katki = v[S | {i}] - v[S]
▶        phi[i] += katki
▶        S = S | {i}
10   return {i: phi[i] / len(siralar) for i in N}
```

DEĞİŞKENLER

`sira = (C, A, B)` `S = {C}` `i = C` `katki = 30` `phi[A] = 100` `phi[B] = 140` `phi[C] = 270`

Kod İzleme: `shapley(N, v)` (2/2)



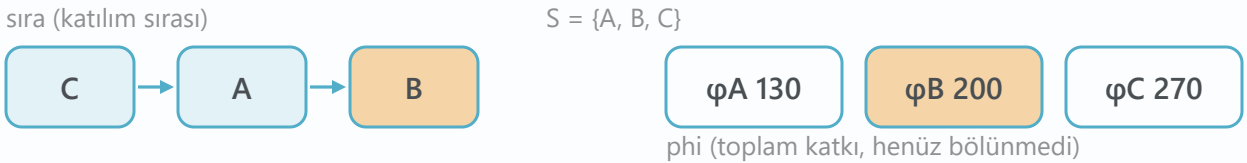
A katılır: katkı = v(AC) − v(C) = 60 − 30 = **30**; phi[A] = 130.


```
1  def shapley(N, v):
2    phi = {i: 0 for i in N}
3    siralar = list(permutations(N))
4    for sira in siralar:
5      S = frozenset()
▶      for i in sira:
▶        katki = v[S | {i}] - v[S]
▶        phi[i] += katki
▶        S = S | {i}
10   return {i: phi[i] / len(siralar) for i in N}
```

DEĞİŞKENLER

`sira = (C, A, B)` `S = {A, C}` `i = A` `katki = 30` `phi[A] = 130` `phi[B] = 140` `phi[C] = 270`

Kod İzleme: `shapley(N, v)` (2/2)



B katılır: katkı = v(ABC) − v(AC) = 120 − 60 = **60**; phi[B] = 200.

Adım 8 / 13

```
1  def shapley(N, v):
2    phi = {i: 0 for i in N}
3    siralar = list(permutations(N))
4    for sira in siralar:
5      S = frozenset()
▶      for i in sira:
▶        katki = v[S | {i}] - v[S]
▶        phi[i] += katki
▶        S = S | {i}
10   return {i: phi[i] / len(siralar) for i in N}
```

DEĞİŞKENLER

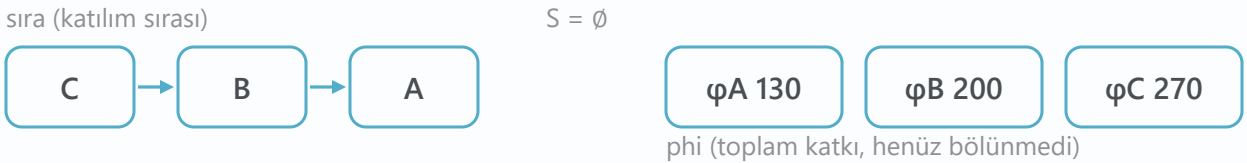
`sira = (C, A, B)`

`S = {A, B, C}` `i = B`

`katki = 60` `phi[A] = 130`

`phi[B] = 200` `phi[C] = 270`

Kod İzleme: `shapley(N, v)` (2/2)



6. sıra (C, B, A): ajanlar bu sırayla gelir. S boş koalisyonla başlar.

Adım 9 / 13

```
1  def shapley(N, v):
2    phi = {i: 0 for i in N}
3    siralar = list(permutations(N))
▶    for sira in siralar:
▶      S = frozenset()
6      for i in sira:
7        katki = v[S | {i}] - v[S]
8        phi[i] += katki
9        S = S | {i}
10   return {i: phi[i] / len(siralar) for i in N}
```

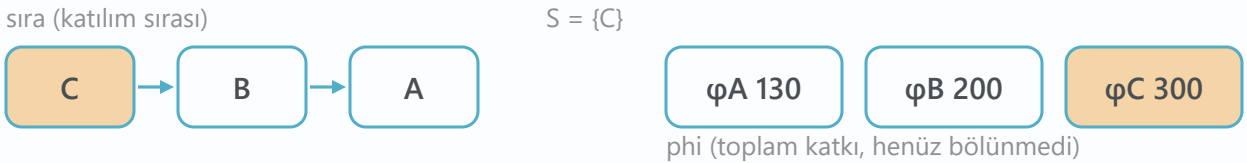
DEĞİŞKENLER

`sira = (C, B, A)` `S = ∅`

`phi[A] = 130` `phi[B] = 200`

`phi[C] = 270`

Kod İzleme: `shapley(N, v)` (2/2)



C katılır: katkı = v(C) − v(∅) = 30 − 0 = **30**; phi[C] = 300.

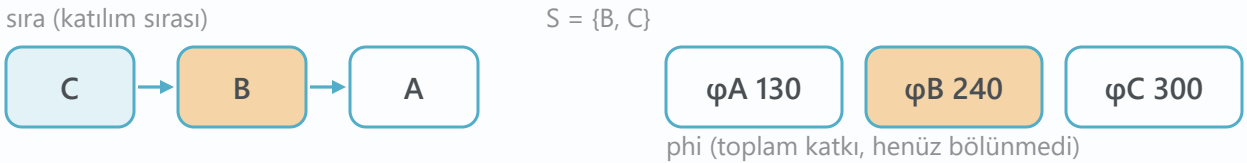
Adım 10 / 13

```
1  def shapley(N, v):
2    phi = {i: 0 for i in N}
3    siralar = list(permutations(N))
4    for sira in siralar:
5      S = frozenset()
▶      for i in sira:
▶        katki = v[S | {i}] - v[S]
▶        phi[i] += katki
▶        S = S | {i}
10   return {i: phi[i] / len(siralar) for i in N}
```

DEĞİŞKENLER

`sira = (C, B, A)` `S = {C}` `i = C` `katki = 30` `phi[A] = 130` `phi[B] = 200` `phi[C] = 300`

Kod İzleme: `shapley(N, v)` (2/2)



B katılır: katkı = v(BC) − v(C) = 70 − 30 = **40**; phi[B] = 240.

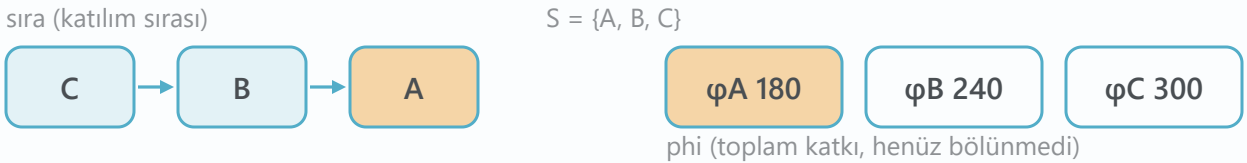
Adım 11 / 13

```
1  def shapley(N, v):
2    phi = {i: 0 for i in N}
3    siralar = list(permutations(N))
4    for sira in siralar:
5      S = frozenset()
▶      for i in sira:
▶        katki = v[S | {i}] - v[S]
▶        phi[i] += katki
▶        S = S | {i}
10   return {i: phi[i] / len(siralar) for i in N}
```

DEĞİŞKENLER

`sira = (C, B, A)` `S = {B, C}` `i = B` `katki = 40` `phi[A] = 130` `phi[B] = 240` `phi[C] = 300`

Kod İzleme: `shapley(N, v)` (2/2)



A katılır: katkı = v(ABC) − v(BC) = 120 − 70 = **50**; phi[A] = 180.

Adım 12 / 13

```
1  def shapley(N, v):
2    phi = {i: 0 for i in N}
3    siralar = list(permutations(N))
4    for sira in siralar:
5      S = frozenset()
▶      for i in sira:
▶        katki = v[S | {i}] - v[S]
▶        phi[i] += katki
▶        S = S | {i}
10   return {i: phi[i] / len(siralar) for i in N}
```

DEĞİŞKENLER

`sira = (C, B, A)`

`S = {A, B, C}` `i = A`

`katki = 50` `phi[A] = 180`

`phi[B] = 240` `phi[C] = 300`

Kod İzleme: `shapley(N, v)` (2/2)

sıra (katılım sırası)

S = ∅

φA 30	φB 40	φC 50

phi (toplam katkı, henüz bölünmedi)

Toplamlar 6'ya bölünür: φA = 180/6 = 30, φB = 240/6 = 40, φC = 300/6 = 50. Toplam 120 = v(ABC).

Adım 13 / 13

```
1 def shapley(N, v):
2   phi = {i: 0 for i in N}
3   siralar = list(permutations(N))
4   for sira in siralar:
5     S = frozenset()
6     for i in sira:
7       katki = v[S | {i}] - v[S]
8       phi[i] += katki
9       S = S | {i}
▶   return {i: phi[i] / len(siralar) for i in N}
```

DEĞİŞKENLER

`phi[A] = 180` `phi[B] = 240` `phi[C] = 300`

`sonuç = {A: 30, B: 40, C: 50}`

Çekirdek (Core)

- x paylaşımı **çekirdektedir** $\Leftrightarrow \Sigma_{i\in N}\, x_i = v(N)$ ve her $S \subseteq N$ için $\Sigma_{i\in S}\, x_i \geq v(S)$.
- Yani hiçbir koalisyon büyük koalisyondan **ayrılarak** (block) üyelerine daha fazlasını sağlayamaz.
- Çekirdek bir doğrusal eşitsizlikler sistemidir (Gillies, 1959): bir çokyüzlü; boş, tek nokta ya da bölge olabilir.

Tekli kısıtlar

$x_A \geq 10$, $x_B \geq 20$, $x_C \geq 30$

İkili kısıtlar

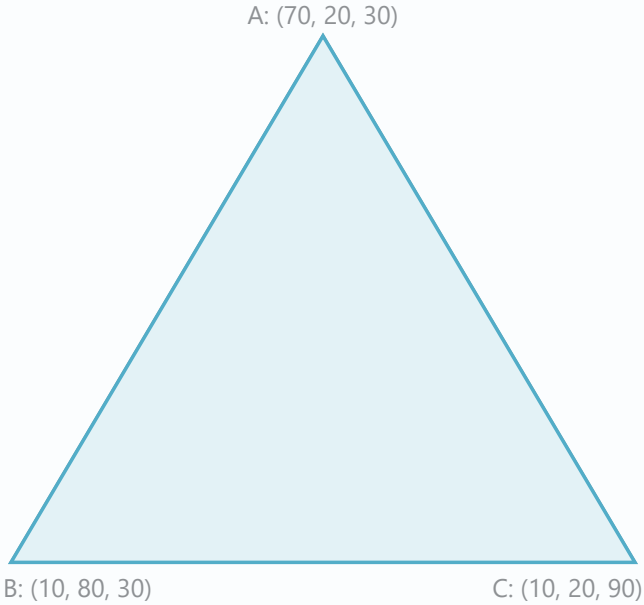
$x_A+x_B \geq 50$, $x_A+x_C \geq 60$, $x_B+x_C \geq 70$

Verimlilik

$x_A+x_B+x_C = 120$

Shapley adalet, çekirdek kararlılık ölçütüdür; ikisi her zaman örtüşmez. **Dışbükey** (convex) oyunlarda (örneğimiz öyle) Shapley değeri çekirdektedir (Shapley, 1971).

Çekirdeğin Geometrisi Adım Adım



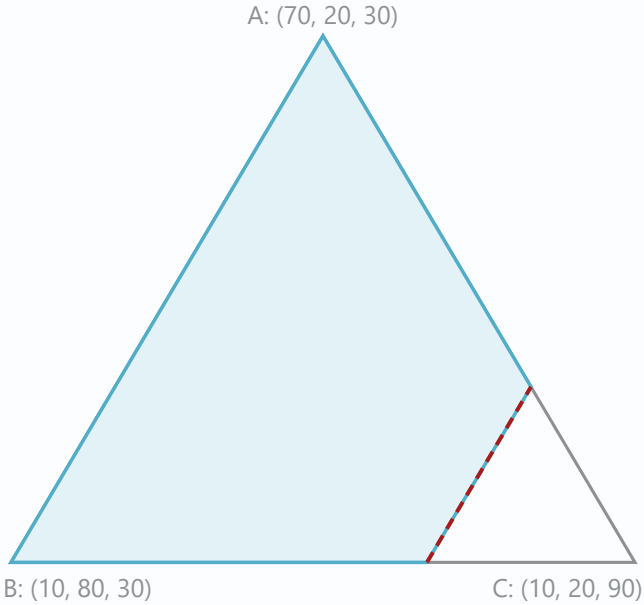
paylaşım x = (xA, xB, xC), toplam 120

üçgen: xA ≥ 10, xB ≥ 20, xC ≥ 30

- A+B ≥ 50 ⇔ xC ≤ 70
- A+C ≥ 60 ⇔ xB ≤ 60
- B+C ≥ 70 ⇔ xA ≤ 50

Bireysel rasyonel ve verimli tüm paylaşımlar bir üçgendir; köşede bir ajan artan 60'ın hepsini alır.

Çekirdeğin Geometrisi Adım Adım



paylaşım x = (xA, xB, xC), toplam 120

üçgen: xA ≥ 10, xB ≥ 20, xC ≥ 30

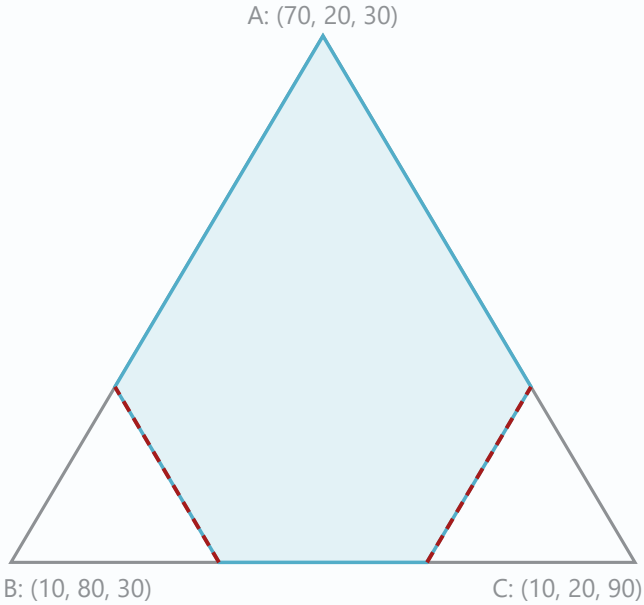
✓ A+B ≥ 50 ⇔ xC ≤ 70

· A+C ≥ 60 ⇔ xB ≤ 60

· B+C ≥ 70 ⇔ xA ≤ 50

{A, B}: $x_A + x_B \geq 50 \Leftrightarrow x_C \leq 70$: C'ye 70'ten fazlası verilirse A ve B ayrılıp kendi 50'lerini alır. C köşesi kesilir.

Çekirdeğin Geometrisi Adım Adım



paylaşım x = (xA, xB, xC), toplam 120

üçgen: xA ≥ 10, xB ≥ 20, xC ≥ 30

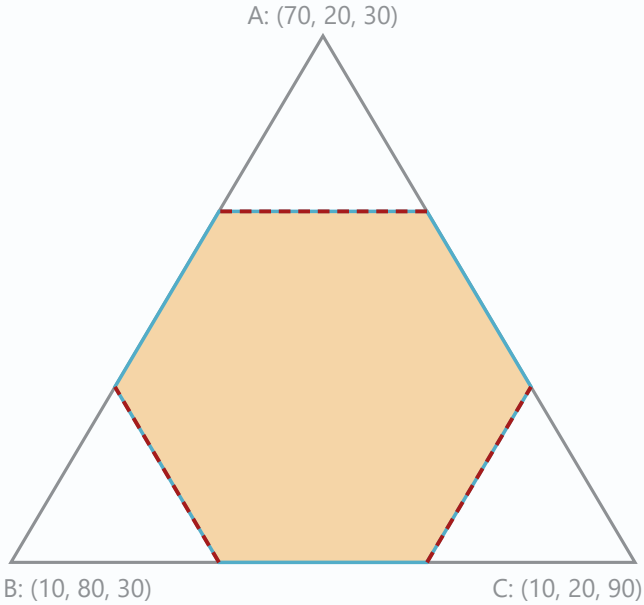
✓ A+B ≥ 50 ⇔ xC ≤ 70

✓ A+C ≥ 60 ⇔ xB ≤ 60

· B+C ≥ 70 ⇔ xA ≤ 50

{A, C}: $x_A + x_C \geq 60 \Leftrightarrow x_B \leq 60$: B köşesi kesilir.

Çekirdeğin Geometrisi Adım Adım



paylaşım x = (xA, xB, xC), toplam 120

üçgen: xA ≥ 10, xB ≥ 20, xC ≥ 30

✓ A+B ≥ 50 ⇔ xC ≤ 70

✓ A+C ≥ 60 ⇔ xB ≤ 60

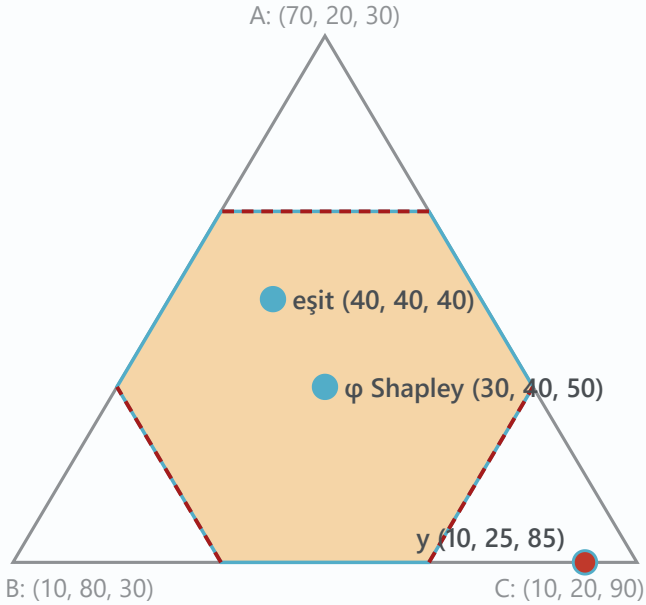
✓ B+C ≥ 70 ⇔ xA ≤ 50

çekirdek: 6 köşeli çokgen

köşeler: (10, 40, 70), (10, 60, 50), (30, 20, 70)
(30, 60, 30), (50, 20, 50), (50, 40, 30)

{B, C}: $x_B + x_C \geq 70 \Leftrightarrow x_A \leq 50$: A köşesi de kesilir. Kalan turuncu altıgen **çekirdektir**.

Çekirdeğin Geometrisi Adım Adım



paylaşım x = (xA, xB, xC), toplam 120

üçgen: xA ≥ 10, xB ≥ 20, xC ≥ 30

✓ **A+B ≥ 50 ⇔ xC ≤ 70**

✓ **A+C ≥ 60 ⇔ xB ≤ 60**

✓ **B+C ≥ 70 ⇔ xA ≤ 50**

çekirdek: 6 köşeli çokgen

köşeler: (10, 40, 70), (10, 60, 50), (30, 20, 70)
(30, 60, 30), (50, 20, 50), (50, 40, 30)

φ = (30, 40, 50) ve eşit bölüşüm çekirdekte (mavi). y = (10, 25, 85) dışarıda (kırmızı): $x_A + x_B = 35 < 50$.

Çekirdek Kontrolü Adım Adım

koalisyon S	kısıt	x = (30, 40, 50)	y = (10, 25, 85)
{A}	Σ ≥ 10	·	·
{B}	Σ ≥ 20	·	·
{C}	Σ ≥ 30	·	·
{A, B}	Σ ≥ 50	·	·
{A, C}	Σ ≥ 60	·	·
{B, C}	Σ ≥ 70	·	·
{A, B, C}	toplam = 120	·	·

Bir paylaşım çekirdekteyse **hiçbir koalisyon** ayrılıp kendi değerini (v(S)) alarak daha iyi duruma geçemez. İki paylaşımı 7 kısıtla sınayalım.

Çekirdek Kontrolü Adım Adım

koalisyon S	kısıt	x = (30, 40, 50)	y = (10, 25, 85)
{A}	Σ ≥ 10	30 ✓	10 ✓
{B}	Σ ≥ 20	·	·
{C}	Σ ≥ 30	·	·
{A, B}	Σ ≥ 50	·	·
{A, C}	Σ ≥ 60	·	·
{B, C}	Σ ≥ 70	·	·
{A, B, C}	toplam = 120	·	·

{A}: v(A) = 10. x için 30 ≥ 10 ✓; y için 10 ≥ 10 ✓.

Çekirdek Kontrolü Adım Adım

koalisyon S	kısıt	x = (30, 40, 50)	y = (10, 25, 85)
{A}	Σ ≥ 10	30 ✓	10 ✓
{B}	Σ ≥ 20	40 ✓	25 ✓
{C}	Σ ≥ 30	·	·
{A, B}	Σ ≥ 50	·	·
{A, C}	Σ ≥ 60	·	·
{B, C}	Σ ≥ 70	·	·
{A, B, C}	toplam = 120	·	·

{B}: v(B) = 20. x için 40 ≥ 20 ✓; y için 25 ≥ 20 ✓.

Çekirdek Kontrolü Adım Adım

koalisyon S	kısıt	x = (30, 40, 50)	y = (10, 25, 85)
{A}	Σ ≥ 10	30 ✓	10 ✓
{B}	Σ ≥ 20	40 ✓	25 ✓
{C}	Σ ≥ 30	50 ✓	85 ✓
{A, B}	Σ ≥ 50	·	·
{A, C}	Σ ≥ 60	·	·
{B, C}	Σ ≥ 70	·	·
{A, B, C}	toplam = 120	·	·

{C}: v(C) = 30. x için 50 ≥ 30 ✓; y için 85 ≥ 30 ✓.

Çekirdek Kontrolü Adım Adım

koalisyon S	kısıt	x = (30, 40, 50)	y = (10, 25, 85)
{A}	Σ ≥ 10	30 ✓	10 ✓
{B}	Σ ≥ 20	40 ✓	25 ✓
{C}	Σ ≥ 30	50 ✓	85 ✓
{A, B}	Σ ≥ 50	70 ✓	35 ✗
{A, C}	Σ ≥ 60	·	·
{B, C}	Σ ≥ 70	·	·
{A, B, C}	toplam = 120	·	·

{A, B}: v(AB) = 50. x için 70 ≥ 50 ✓; y için 35 ≥ 50 ✗. {A, B} birlikte ayrılırsa 50 alır: y'yi **engeller** (block).

Çekirdek Kontrolü Adım Adım

koalisyon S	kısıt	x = (30, 40, 50)	y = (10, 25, 85)
{A}	Σ ≥ 10	30 ✓	10 ✓
{B}	Σ ≥ 20	40 ✓	25 ✓
{C}	Σ ≥ 30	50 ✓	85 ✓
{A, B}	Σ ≥ 50	70 ✓	35 ✗
{A, C}	Σ ≥ 60	80 ✓	95 ✓
{B, C}	Σ ≥ 70	·	·
{A, B, C}	toplam = 120	·	·

{A, C}: v(AC) = 60. x için 80 ≥ 60 ✓; y için 95 ≥ 60 ✓.

Çekirdek Kontrolü Adım Adım

koalisyon S	kısıt	x = (30, 40, 50)	y = (10, 25, 85)
{A}	Σ ≥ 10	30 ✓	10 ✓
{B}	Σ ≥ 20	40 ✓	25 ✓
{C}	Σ ≥ 30	50 ✓	85 ✓
{A, B}	Σ ≥ 50	70 ✓	35 ✗
{A, C}	Σ ≥ 60	80 ✓	95 ✓
{B, C}	Σ ≥ 70	90 ✓	110 ✓
{A, B, C}	toplam = 120	·	·

{B, C}: v(BC) = 70. x için 90 ≥ 70 ✓; y için 110 ≥ 70 ✓.

Çekirdek Kontrolü Adım Adım

koalisyon S	kısıt	x = (30, 40, 50)	y = (10, 25, 85)
{A}	Σ ≥ 10	30 ✓	10 ✓
{B}	Σ ≥ 20	40 ✓	25 ✓
{C}	Σ ≥ 30	50 ✓	85 ✓
{A, B}	Σ ≥ 50	70 ✓	35 ✗
{A, C}	Σ ≥ 60	80 ✓	95 ✓
{B, C}	Σ ≥ 70	90 ✓	110 ✓
{A, B, C}	toplam = 120	120 ✓	120 ✓

{A, B, C}: v(ABC) = 120. x için 120 = 120 ✓; y için 120 = 120 ✓.

Çekirdek Kontrolü Adım Adım

koalisyon S	kısıt	x = (30, 40, 50)	y = (10, 25, 85)
{A}	Σ ≥ 10	30 ✓	10 ✓
{B}	Σ ≥ 20	40 ✓	25 ✓
{C}	Σ ≥ 30	50 ✓	85 ✓
{A, B}	Σ ≥ 50	70 ✓	35 ✗
{A, C}	Σ ≥ 60	80 ✓	95 ✓
{B, C}	Σ ≥ 70	90 ✓	110 ✓
{A, B, C}	toplam = 120	120 ✓	120 ✓

Sonuç: Shapley paylaşımı x **çekirdekte**; y ise {A, B} tarafından engellendiği için **çekirdek dışında**. (Oyun dışbükey olduğundan Shapley değeri çekirdektedir.)

Çekirdeği Boş Bir Oyun: Çoğunluk Oyunu

- Üç ajan 1 birimlik ödülü paylaşacak; herhangi **iki** ajan anlaşırsa ödülü alır: $v(S) = 1$ ($|S| \geq 2$), aksi hâlde 0.
- Çekirdek kısıtları: $x_A+x_B \geq 1$, $x_A+x_C \geq 1$, $x_B+x_C \geq 1$.
- Toplayınca $2(x_A+x_B+x_C) \geq 3$, yani toplam $\geq 3/2$. Ama toplam = 1 olmalı ⇒ **çekirdek boş**.
- Her paylaşımda en az bir çift $\leq 2/3$ alır ve üçüncüyü dışlayıp 1'i paylaşmak ister: sonsuz pazarlık.

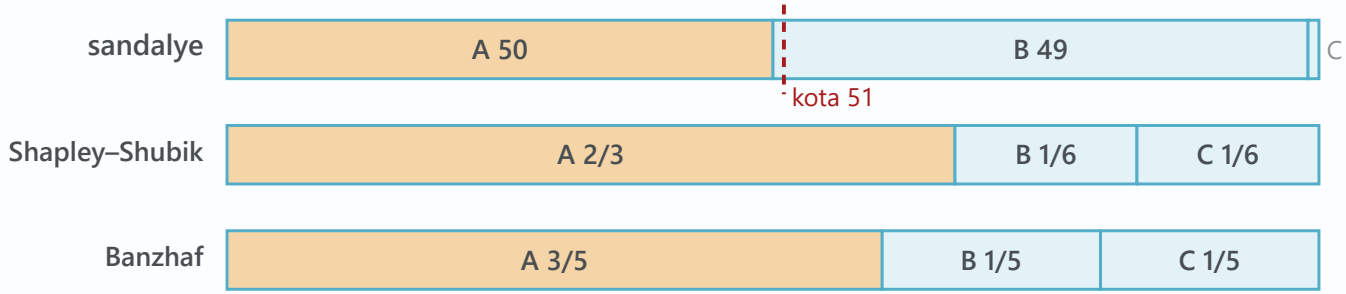
Shapley

$\varphi = (1/3, 1/3, 1/3)$: adil ama kararlı değil.

Ders

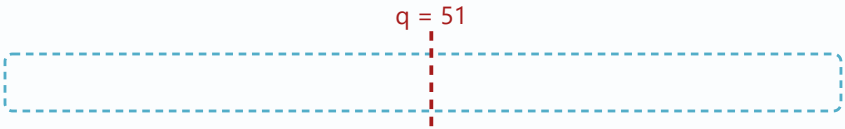
Kararlı paylaşım her oyunda yoktur; o zaman ε-çekirdek, çekirdekçik (nucleolus) gibi zayıflatılmış kavramlar kullanılır.

Ağırlıklı Oylama: Ağırlık ≠ Güç



- **Ağırlıklı oylama oyunu** [q; w_1, …, w_n]: koalisyon ağırlıkları toplamı q'ya ulaşırsa kazanır, v(S) = 1; yoksa 0.
- **Shapley–Shubik endeksi** (1954) = bu oyunun Shapley değeri: i'nin **kilit** (pivotal) olduğu sıraların oranı.
- Meclis [51; 50, 49, 1]: B ile C, 48 sandalye farkına rağmen **eşit** güçtedir (1/6); Banzhaf endeksi de (3/5, 1/5, 1/5).
- 1958 AET Bakanlar Konseyi [12; 4, 4, 4, 2, 2, 1]: Lüksemburg'un oyu hiçbir koalisyonun sonucunu değiştiremiyordu, güç = 0.

Kod İzleme: `guc_endeksi(N, w, q)`



kilit sayacı

A 0

B 0

C 0

w = {A: 50, B: 49, C: 1}, q = 51. 3! = 6 geliş sırası; her sırada kotayı geçiren oyuncu kilittir.

Adım 1 / 14

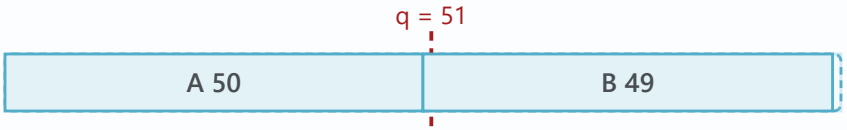
```
1  def guc_endeksi(N, w, q):
▶    kilit = {i: 0 for i in N}
▶    siralar = list(permutations(N))
4    for sira in siralar:
5      toplam = 0
6      for i in sira:
7        toplam += w[i]
8        if toplam >= q:
9          kilit[i] += 1
10         break
11   return {i: kilit[i] / len(siralar) for i in N}
```

DEĞİŞKENLER

`kilit = {A: 0, B: 0, C: 0}`

`siralar = 6 sıra`

Kod İzleme: `guc_endeksi(N, w, q)`



sıra: A → B → C

kilit sayacı

A 0

B 0

C 0

Sıra A → B → C: toplam 50 < 51 iken B gelir, toplam 99 ≥ 51.

Adım 2 / 14

```
1  def guc_endeksi(N, w, q):
2    kilit = {i: 0 for i in N}
3    siralar = list(permutations(N))
▶    for sira in siralar:
▶      toplam = 0
▶      for i in sira:
▶        toplam += w[i]
8        if toplam >= q:
9          kilit[i] += 1
10         break
11   return {i: kilit[i] / len(siralar) for i in N}
```

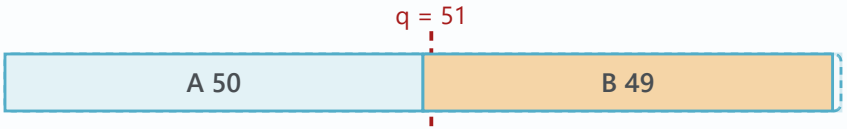
DEĞİŞKENLER

`sira = (A, B, C)` `i = B`

`toplam = 99`

`kilit = {A: 0, B: 1, C: 0}`

Kod İzleme: `guc_endeksi(N, w, q)`



kilit sayacı

A 0

B 1

C 0

sıra: A → B → C kilit: B

B kilit: kilit[B] = 1; `break` ile sıradaki geliş sırasına geçilir.

Adım 3 / 14

```
1  def guc_endeksi(N, w, q):
2    kilit = {i: 0 for i in N}
3    siralar = list(permutations(N))
4    for sira in siralar:
5      toplam = 0
6      for i in sira:
7        toplam += w[i]
▶        if toplam >= q:
▶          kilit[i] += 1
▶          break
11   return {i: kilit[i] / len(siralar) for i in N}
```

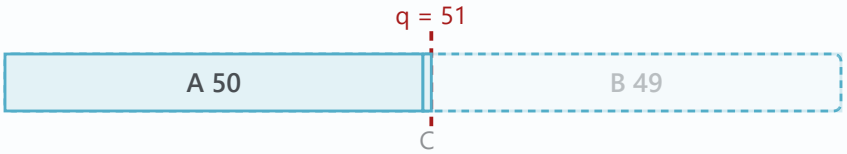
DEĞİŞKENLER

`sira = (A, B, C)` `i = B`

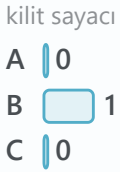
`toplam = 99`

`kilit = {A: 0, B: 1, C: 0}`

Kod İzleme: `guc_endeksi(N, w, q)`



sıra: A → C → B



Sıra A → C → B: toplam 50 < 51 iken C gelir, toplam 51 ≥ 51.

Adım 4 / 14

```
1  def guc_endeksi(N, w, q):
2    kilit = {i: 0 for i in N}
3    siralar = list(permutations(N))
▶    for sira in siralar:
▶      toplam = 0
▶      for i in sira:
▶        toplam += w[i]
8        if toplam >= q:
9          kilit[i] += 1
10         break
11   return {i: kilit[i] / len(siralar) for i in N}
```

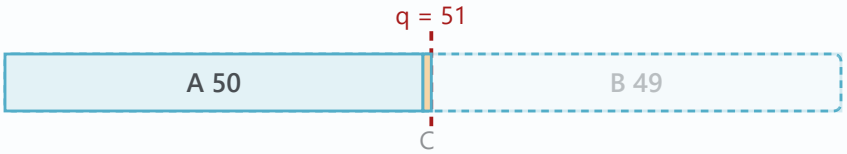
DEĞİŞKENLER

`sira = (A, C, B)` `i = C`

`toplam = 51`

`kilit = {A: 0, B: 1, C: 1}`

Kod İzleme: `guc_endeksi(N, w, q)`



kilit sayacı

A 0

B 1

C 1

sıra: A → C → B kilit: C

C kilit: kilit[C] = 1; `break` ile sıradaki geliş sırasına geçilir.

Adım 5 / 14

```
1  def guc_endeksi(N, w, q):
2    kilit = {i: 0 for i in N}
3    siralar = list(permutations(N))
4    for sira in siralar:
5      toplam = 0
6      for i in sira:
7        toplam += w[i]
▶        if toplam >= q:
▶          kilit[i] += 1
▶          break
11   return {i: kilit[i] / len(siralar) for i in N}
```

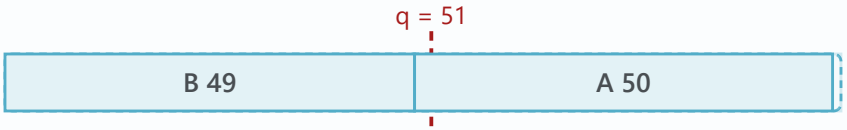
DEĞİŞKENLER

`sira = (A, C, B)` `i = C`

`toplam = 51`

`kilit = {A: 0, B: 1, C: 1}`

Kod İzleme: `guc_endeksi(N, w, q)`



sıra: B → A → C

kilit sayacı

A 0

B 1

C 1

Sıra B → A → C: toplam 49 < 51 iken A gelir, toplam 99 ≥ 51.

Adım 6 / 14

```
1  def guc_endeksi(N, w, q):
2    kilit = {i: 0 for i in N}
3    siralar = list(permutations(N))
▶    for sira in siralar:
▶      toplam = 0
▶      for i in sira:
▶        toplam += w[i]
8        if toplam >= q:
9          kilit[i] += 1
10         break
11   return {i: kilit[i] / len(siralar) for i in N}
```

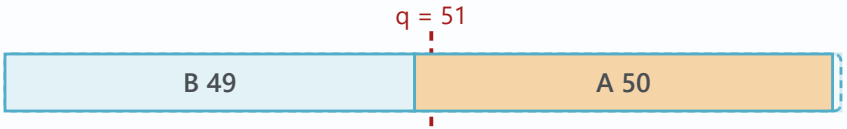
DEĞİŞKENLER

`sira = (B, A, C)` `i = A`

`toplam = 99`

`kilit = {A: 1, B: 1, C: 1}`

Kod İzleme: `guc_endeksi(N, w, q)`



kilit sayacı

A	1
B	1
C	1

sıra: B → A → C kilit: A

A kilit: kilit[A] = 1; `break` ile sıradaki geliş sırasına geçilir.

Adım 7 / 14

```
1  def guc_endeksi(N, w, q):
2    kilit = {i: 0 for i in N}
3    siralar = list(permutations(N))
4    for sira in siralar:
5      toplam = 0
6      for i in sira:
7        toplam += w[i]
▶        if toplam >= q:
▶          kilit[i] += 1
▶          break
11   return {i: kilit[i] / len(siralar) for i in N}
```

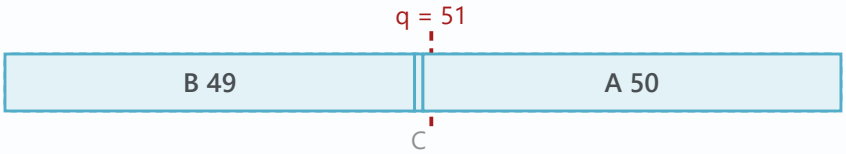
DEĞİŞKENLER

`sira = (B, A, C)` `i = A`

`toplam = 99`

`kilit = {A: 1, B: 1, C: 1}`

Kod İzleme: `guc_endeksi(N, w, q)`



sıra: B → C → A

kilit sayacı

A	1
B	1
C	1

Sıra B → C → A: toplam 50 < 51 iken A gelir, toplam 100 ≥ 51.

Adım 8 / 14

```
1  def guc_endeksi(N, w, q):
2    kilit = {i: 0 for i in N}
3    siralar = list(permutations(N))
▶    for sira in siralar:
▶      toplam = 0
▶      for i in sira:
▶        toplam += w[i]
8        if toplam >= q:
9          kilit[i] += 1
10         break
11   return {i: kilit[i] / len(siralar) for i in N}
```

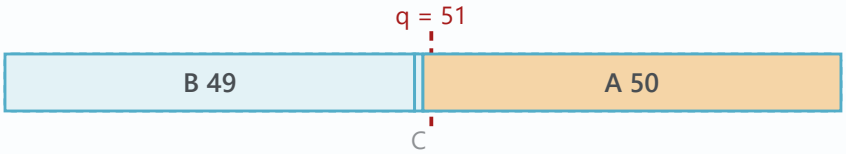
DEĞİŞKENLER

`sira = (B, C, A)` `i = A`

`toplam = 100`

`kilit = {A: 2, B: 1, C: 1}`

Kod İzleme: `guc_endeksi(N, w, q)`



sıra: B → C → A kilit: A



A kilit: kilit[A] = 2; `break` ile sıradaki geliş sırasına geçilir.

Adım 9 / 14

```
1  def guc_endeksi(N, w, q):
2    kilit = {i: 0 for i in N}
3    siralar = list(permutations(N))
4    for sira in siralar:
5      toplam = 0
6      for i in sira:
7        toplam += w[i]
▶        if toplam >= q:
▶          kilit[i] += 1
▶          break
11   return {i: kilit[i] / len(siralar) for i in N}
```

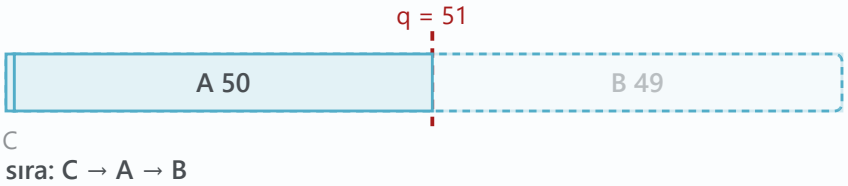
DEĞİŞKENLER

`sira = (B, C, A)` `i = A`

`toplam = 100`

`kilit = {A: 2, B: 1, C: 1}`

Kod İzleme: `guc_endeksi(N, w, q)`



kilit sayacı

A	2
B	1
C	1

Sıra C → A → B: toplam 1 < 51 iken A gelir, toplam 51 ≥ 51.

Adım 10 / 14

```
1  def guc_endeksi(N, w, q):
2    kilit = {i: 0 for i in N}
3    siralar = list(permutations(N))
▶    for sira in siralar:
▶      toplam = 0
▶      for i in sira:
▶        toplam += w[i]
8        if toplam >= q:
9          kilit[i] += 1
10         break
11   return {i: kilit[i] / len(siralar) for i in N}
```

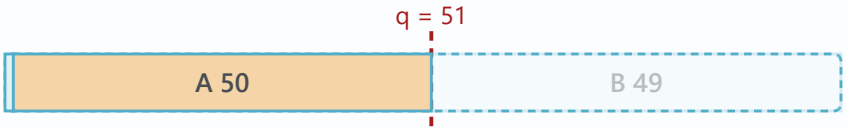
DEĞİŞKENLER

`sira = (C, A, B)` `i = A`

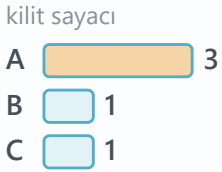
`toplam = 51`

`kilit = {A: 3, B: 1, C: 1}`

Kod İzleme: `guc_endeksi(N, w, q)`



sıra: C → A → B kilit: A



A kilit: kilit[A] = 3; `break` ile sıradaki geliş sırasına geçilir.

Adım 11 / 14

```
1  def guc_endeksi(N, w, q):
2    kilit = {i: 0 for i in N}
3    siralar = list(permutations(N))
4    for sira in siralar:
5      toplam = 0
6      for i in sira:
7        toplam += w[i]
▶        if toplam >= q:
▶          kilit[i] += 1
▶          break
11   return {i: kilit[i] / len(siralar) for i in N}
```

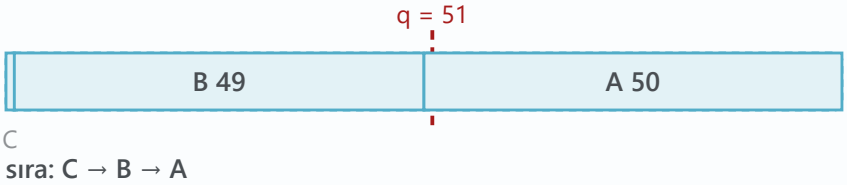
DEĞİŞKENLER

`sira = (C, A, B)` `i = A`

`toplam = 51`

`kilit = {A: 3, B: 1, C: 1}`

Kod İzleme: `guc_endeksi(N, w, q)`





Sıra C → B → A: toplam 50 < 51 iken A gelir, toplam 100 ≥ 51.

Adım 12 / 14

```
1  def guc_endeksi(N, w, q):
2    kilit = {i: 0 for i in N}
3    siralar = list(permutations(N))
▶    for sira in siralar:
▶      toplam = 0
▶      for i in sira:
▶        toplam += w[i]
8        if toplam >= q:
9          kilit[i] += 1
10         break
11   return {i: kilit[i] / len(siralar) for i in N}
```

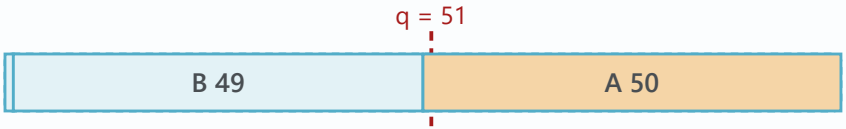
DEĞİŞKENLER

`sira = (C, B, A)` `i = A`

`toplam = 100`

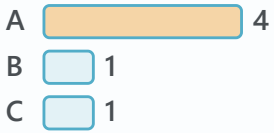
`kilit = {A: 4, B: 1, C: 1}`

Kod İzleme: `guc_endeksi(N, w, q)`



sıra: C → B → A kilit: A

kilit sayacı



A kilit: kilit[A] = 4; `break` ile sıradaki geliş sırasına geçilir.

Adım 13 / 14

```
1  def guc_endeksi(N, w, q):
2    kilit = {i: 0 for i in N}
3    siralar = list(permutations(N))
4    for sira in siralar:
5      toplam = 0
6      for i in sira:
7        toplam += w[i]
▶        if toplam >= q:
▶          kilit[i] += 1
▶          break
11   return {i: kilit[i] / len(siralar) for i in N}
```

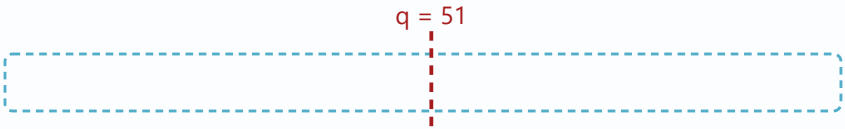
DEĞİŞKENLER

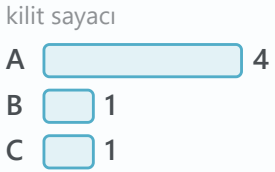
`sira = (C, B, A)` `i = A`

`toplam = 100`

`kilit = {A: 4, B: 1, C: 1}`

Kod İzleme: `guc_endeksi(N, w, q)`





Güç = kilit / 6 ⇒ (2/3, 1/6, 1/6). B'nin 49 sandalyesi C'nin 1 sandalyesinden fazla güç getirmiyor.

Adım 14 / 14

```
1  def guc_endeksi(N, w, q):
2    kilit = {i: 0 for i in N}
3    siralar = list(permutations(N))
4    for sira in siralar:
5      toplam = 0
6      for i in sira:
7        toplam += w[i]
8        if toplam >= q:
9          kilit[i] += 1
10         break
▶    return {i: kilit[i] / len(siralar) for i in N}
```

DEĞİŞKENLER

`kilit = {A: 4, B: 1, C: 1}`

`sonuc = {A: 0.667, B: 0.167, C: 0.167}`

Kamu Malı Oyunu (Public Goods Game)

- **Kamu malı**: kimse kullanımdan dışlanamaz ve bir kişinin kullanımı diğerininkini azaltmaz (ör. açık kaynak yazılım, temiz hava, ortak veri kümesi).
- Oyun: n oyuncu, her biri e jeton. Oyuncu i ortak havuza $c_i \in [0, e]$ katar. Havuz r çarpanıyla büyür ve **herkese eşit** dağıtılır.
- Kazanç: $u_i = e - c_i + (r/n) \cdot \Sigma_j \, c_j$
- r/n: bir jetonun katkı yapana geri dönen kısmı (MPCR). **1/n < r/n < 1** ise ikilem vardır.

Örneğimiz

n = 4, e = 20, r = 1,6 ⇒ r/n = 0,4

Bireysel

1 jeton katkı: kendine 0,4 döner ⇒ net −0,6

Sosyal

1 jeton katkı: gruba 1,6 döner ⇒ net +0,6

Kamu Malı Oyunu: Dört Senaryo

senaryo	katkılar	havuz × 1,6	kazançlar (O1, O2, O3, O4)	toplam
Herkes tam katkı	·	·	·	·
Kimse katkı vermez	·	·	·	·
Bir beleşçi	·	·	·	·
Tek katkıcı	·	·	·	·

Her oyuncu 20 jetonla başlar. Katkıların toplamı 1,6 ile çarpılır ve **4 kişiye eşit** dağıtılır: $u_i = 20 - c_i + 0{,}4 \cdot \Sigma c$.

Kamu Malı Oyunu: Dört Senaryo

senaryo	katkılar	havuz × 1,6	kazançlar (O1, O2, O3, O4)	toplam
Herkes tam katkı	(20, 20, 20, 20)	128	(32, 32, 32, 32)	128
Kimse katkı vermez	·	·	·	·
Bir beleşçi	·	·	·	·
Tek katkıcı	·	·	·	·

Herkes 20 verir: havuz 80 × 1,6 = 128; herkes 32 alır. Toplam refah en yüksek: 128.

Kamu Malı Oyunu: Dört Senaryo

senaryo	katkılar	havuz × 1,6	kazançlar (O1, O2, O3, O4)	toplam
Herkes tam katkı	(20, 20, 20, 20)	128	(32, 32, 32, 32)	128
Kimse katkı vermez	(0, 0, 0, 0)	0	(20, 20, 20, 20)	80
Bir beleşçi	·	·	·	·
Tek katkıcı	·	·	·	·

Kimse vermez: herkes 20'sini tutar; toplam 80. Herkes tam katkıya göre 12 jeton kaybeder.

Kamu Malı Oyunu: Dört Senaryo

senaryo	katkılar	havuz × 1,6	kazançlar (O1, O2, O3, O4)	toplam
Herkes tam katkı	(20, 20, 20, 20)	128	(32, 32, 32, 32)	128
Kimse katkı vermez	(0, 0, 0, 0)	0	(20, 20, 20, 20)	80
Bir beleşçi	(0, 20, 20, 20)	96	(44, 24, 24, 24)	116
Tek katkıcı	·	·	·	·

O1 beleşçi (0), diğerleri 20: havuz 60 × 1,6 = 96, kişi başı 24. Beleşçi **44** alır — tam katkıdaki 32'den fazla!

Kamu Malı Oyunu: Dört Senaryo

senaryo	katkılar	havuz × 1,6	kazançlar (O1, O2, O3, O4)	toplam
Herkes tam katkı	(20, 20, 20, 20)	128	(32, 32, 32, 32)	128
Kimse katkı vermez	(0, 0, 0, 0)	0	(20, 20, 20, 20)	80
Bir beleşçi	(0, 20, 20, 20)	96	(44, 24, 24, 24)	116
Tek katkıcı	(20, 0, 0, 0)	32	(8, 28, 28, 28)	92

Tek katkıcı: O1 20 verir, havuz 32, kişi başı 8. Katkıcı **8** alır (verdiğinin 12 eksiği); diğerleri 28.

Neden Beleşçilik (Free Riding)?

- $u_1 = 20 - c_1 + 0{,}4\cdot(c_1 + \text{diğerleri}) \Rightarrow \partial u_1/\partial c_1 = -1 + 0{,}4 = \mathbf{-0{,}6}$.
- Diğerleri ne yaparsa yapsın her jeton katkı O1'e 0,6 kaybettirir: **c = 0 baskın strateji**.
- Tek Nash dengesi: herkes 0 → kişi başı 20. Oysa herkes 20 verse kişi başı 32: denge **Pareto optimal değil**.
- Kırmızı doğru: diğerleri tam katkı verirken O1'in kazancı 44'ten 32'ye düşer. Mavi: herkes aynı c'yi verirse kişi başı 20'den 32'ye çıkar.



Mahkûmun İkilemi'nin n kişilik hâli: bireysel teşvik (−0,6) ile sosyal refah (+0,6) ters yönde.

KAMU MALI OYUNU

Deneyler ve Beleşçiliğe Karşı Çözümler

Laboratuvar bulguları

- İlk turda katkılar genellikle bütçenin **%40–60**'ı; teori 0 der (Ledyard, 1995).
- Tekrarlanan turlarda katkılar **giderek düşer**: deneklerin yaklaşık yarısı koşullu işbirlikçidir, beleşçilere tepki verir (Fischbacher, Gächter, Fehr, 2001).
- Maliyetli **ceza** imkânı verilince katkılar yüksek kalır (Fehr ve Gächter, 2000).
- MAS örneği: dosya paylaşım ağında yükleme yapmadan yalnızca indirmek.

Ceza / ödül

Katkı vermeyene yaptırım; teşvik gradyanını pozitife çevirir.

Tekrar ve itibar

Uzun ilişkide koşullu işbirliği sürdürülebilir.

Eşik + iade

Hedef toplanmazsa para iade (bağış kampanyaları).

Mekanizma

Katkıyı zorunlu/koşullu kılan kurallar (vergi, kota).

KAMU MALI OYUNU

Kod İzleme: `kosullu_katki(c, tur)`



tur 0: katkılar

[0, 20, 20, 20]

kazanç u = 20 − c + 0,4·Σc

O1: 44 O2: 24

Model: O1 hep 0 verir; O2–O4 **koşullu işbirlikçi**: bu tur, diğerlerinin önceki turdaki ortalaması kadar verir.

Adım 1 / 12

```
1  def kosullu_katki(c, tur):
▶    gecmis = [c]
3    for t in range(tur):
4      yeni = [0]                  # O1 hep beleşçi
5      for i in range(1, len(c)):
6        digerleri = sum(c) - c[i]
7        yeni.append(digerleri / (len(c) - 1))
8      c = yeni
9      gecmis.append(c)
10   return gecmis
```

DEĞİŞKENLER

`c = [0, 20, 20, 20]`

`gecmis = [c]`

Kod İzleme: `kosullu_katki(c, tur)`



tur 0: katkılar

[0, 20, 20, 20]

kazanç u = 20 − c + 0,4·Σc

O1: 44 O2: 24

t = 0: O2 için diğerlerinin toplamı 40 ⇒ ortalama 40 / 3 = **13,3** (O3, O4 de aynı).

Adım 2 / 12

```
1  def kosullu_katki(c, tur):
2      gecmis = [c]
▶      for t in range(tur):
▶          yeni = [0]                  # O1 hep beleşçi
▶          for i in range(1, len(c)):
▶              digerleri = sum(c) - c[i]
▶              yeni.append(digerleri / (len(c) - 1))
8          c = yeni
9          gecmis.append(c)
10     return gecmis
```

DEĞİŞKENLER

`t = 0` `c = [0, 20, 20, 20]` `digerleri = 40` `yeni = [0, 13.3, 13.3, 13.3]`

Kod İzleme: `kosullu_katki(c, tur)`



Tur 1: katkılar [0, 13.3, 13.3, 13.3]. Beleşçi O1, herkesten fazla kazanır.

tur 1: katkılar

[0, 13.3, 13.3, 13.3]

kazanç u = 20 − c + 0,4·Σc

O1: 36 O2: 22,7

Adım 3 / 12

```
1  def kosullu_katki(c, tur):
2    gecmis = [c]
3    for t in range(tur):
4      yeni = [0]                    # O1 hep beleşçi
5      for i in range(1, len(c)):
6        digerleri = sum(c) - c[i]
7        yeni.append(digerleri / (len(c) - 1))
▶      c = yeni
▶      gecmis.append(c)
10   return gecmis
```

DEĞİŞKENLER

`t = 0`

`c = [0, 13.3, 13.3, 13.3]`

`gecmis = 2 tur`

Kod İzleme: `kosullu_katki(c, tur)`



tur 1: katkılar

[0, 13.3, 13.3, 13.3]

kazanç u = 20 − c + 0,4·Σc

O1: 36 O2: 22,7

t = 1: O2 için diğerlerinin toplamı 26,7 ⇒ ortalama 26,7 / 3 = **8,9** (O3, O4 de aynı).

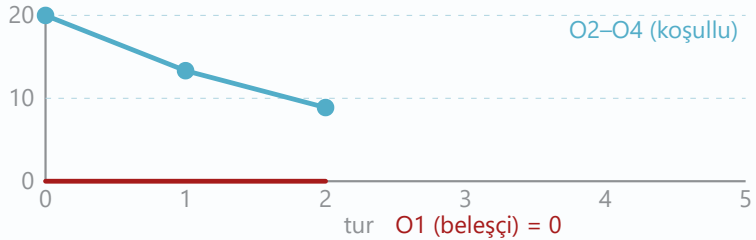
Adım 4 / 12

```
1  def kosullu_katki(c, tur):
2      gecmis = [c]
▶      for t in range(tur):
▶          yeni = [0]                  # O1 hep beleşçi
▶          for i in range(1, len(c)):
▶              digerleri = sum(c) - c[i]
▶              yeni.append(digerleri / (len(c) - 1))
8          c = yeni
9          gecmis.append(c)
10     return gecmis
```

DEĞİŞKENLER

- `t = 1`
- `c = [0, 13.3, 13.3, 13.3]`
- `digerleri = 26.7`
- `yeni = [0, 8.9, 8.9, 8.9]`

Kod İzleme: `kosullu_katki(c, tur)`



tur 2: katkılar
[0, 8.9, 8.9, 8.9]

kazanç u = 20 − c + 0,4·Σc
O1: 30,7 O2: 21,8

Tur 2: katkılar [0, 8.9, 8.9, 8.9]. Her tur ⅔ oranında düşer.

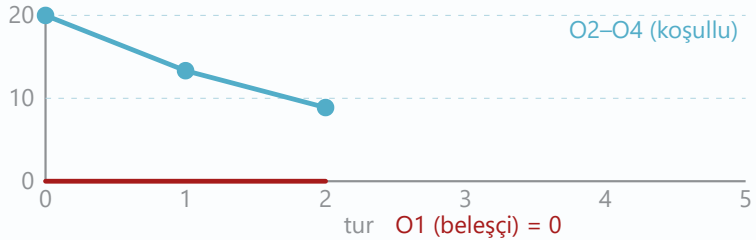
Adım 5 / 12

```
1  def kosullu_katki(c, tur):
2    gecmis = [c]
3    for t in range(tur):
4      yeni = [0]                    # O1 hep beleşçi
5      for i in range(1, len(c)):
6        digerleri = sum(c) - c[i]
7        yeni.append(digerleri / (len(c) - 1))
▶      c = yeni
▶      gecmis.append(c)
10   return gecmis
```

DEĞİŞKENLER

`t = 1` `c = [0, 8.9, 8.9, 8.9]` `gecmis = 3 tur`

Kod İzleme: `kosullu_katki(c, tur)`



tur 2: katkılar
[0, 8.9, 8.9, 8.9]

kazanç u = 20 − c + 0,4·Σc
O1: 30,7 O2: 21,8

t = 2: O2 için diğerlerinin toplamı 17,8 ⇒ ortalama 17,8 / 3 = **5,9** (O3, O4 de aynı).

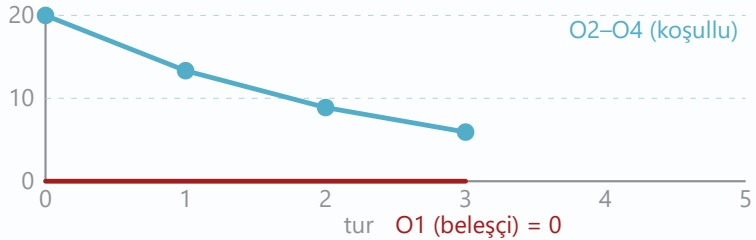
Adım 6 / 12

```
1  def kosullu_katki(c, tur):
2    gecmis = [c]
▶    for t in range(tur):
▶      yeni = [0]                    # O1 hep beleşçi
▶      for i in range(1, len(c)):
▶        digerleri = sum(c) - c[i]
▶        yeni.append(digerleri / (len(c) - 1))
8      c = yeni
9      gecmis.append(c)
10   return gecmis
```

DEĞİŞKENLER

`t = 2` `c = [0, 8.9, 8.9, 8.9]` `digerleri = 17.8` `yeni = [0, 5.9, 5.9, 5.9]`

Kod İzleme: `kosullu_katki(c, tur)`



tur 3: katkılar
[0, 5.9, 5.9, 5.9]

kazanç u = 20 − c + 0,4·Σc
O1: 27,1 O2: 21,2

Tur 3: katkılar [0, 5.9, 5.9, 5.9].

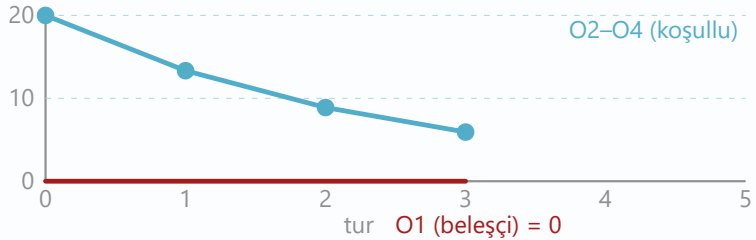
Adım 7 / 12

```
1  def kosullu_katki(c, tur):
2    gecmis = [c]
3    for t in range(tur):
4      yeni = [0]                    # O1 hep beleşçi
5      for i in range(1, len(c)):
6        digerleri = sum(c) - c[i]
7        yeni.append(digerleri / (len(c) - 1))
▶      c = yeni
▶      gecmis.append(c)
10   return gecmis
```

DEĞİŞKENLER

`t = 2` `c = [0, 5.9, 5.9, 5.9]` `gecmis = 4 tur`

Kod İzleme: `kosullu_katki(c, tur)`



tur 3: katkılar

[0, 5.9, 5.9, 5.9]

kazanç u = 20 − c + 0,4·Σc

O1: 27,1 O2: 21,2

t = 3: O2 için diğerlerinin toplamı 11,9 ⇒ ortalama 11,9 / 3 = **4,0** (O3, O4 de aynı).

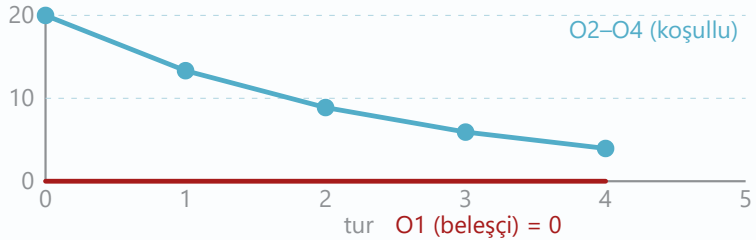
Adım 8 / 12

```
1  def kosullu_katki(c, tur):
2      gecmis = [c]
▶      for t in range(tur):
▶          yeni = [0]                  # O1 hep beleşçi
▶          for i in range(1, len(c)):
▶              digerleri = sum(c) - c[i]
▶              yeni.append(digerleri / (len(c) - 1))
8          c = yeni
9          gecmis.append(c)
10     return gecmis
```

DEĞİŞKENLER

`t = 3` `c = [0, 5.9, 5.9, 5.9]` `digerleri = 11.9` `yeni = [0, 4.0, 4.0, 4.0]`

Kod İzleme: `kosullu_katki(c, tur)`



tur 4: katkılar
[0, 4.0, 4.0, 4.0]

kazanç u = 20 − c + 0,4·Σc
O1: 24,7 O2: 20,8

Tur 4: katkılar [0, 4.0, 4.0, 4.0].

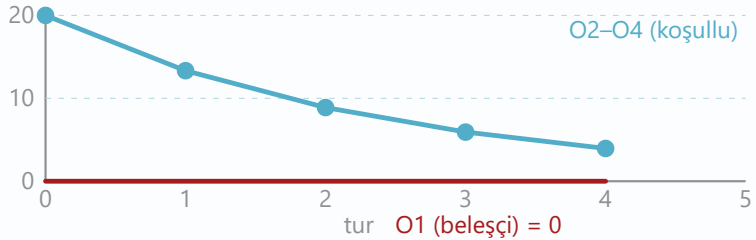
Adım 9 / 12

```
1  def kosullu_katki(c, tur):
2    gecmis = [c]
3    for t in range(tur):
4      yeni = [0]                    # O1 hep beleşçi
5      for i in range(1, len(c)):
6        digerleri = sum(c) - c[i]
7        yeni.append(digerleri / (len(c) - 1))
▶      c = yeni
▶      gecmis.append(c)
10   return gecmis
```

DEĞİŞKENLER

`t = 3` `c = [0, 4.0, 4.0, 4.0]` `gecmis = 5 tur`

Kod İzleme: `kosullu_katki(c, tur)`



tur 4: katkılar

[0, 4.0, 4.0, 4.0]

kazanç u = 20 − c + 0,4·Σc

O1: 24,7 O2: 20,8

t = 4: O2 için diğerlerinin toplamı 7,9 ⇒ ortalama 7,9 / 3 = **2,6** (O3, O4 de aynı).

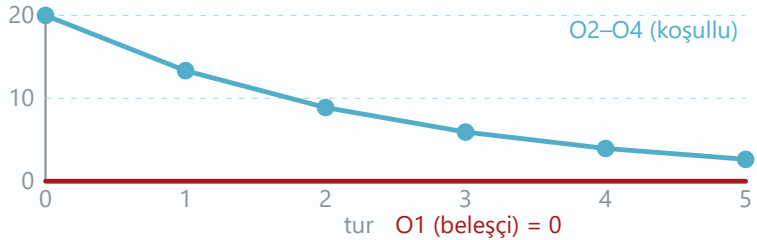
Adım 10 / 12

```
1  def kosullu_katki(c, tur):
2      gecmis = [c]
▶      for t in range(tur):
▶          yeni = [0]                  # O1 hep beleşçi
▶          for i in range(1, len(c)):
▶              digerleri = sum(c) - c[i]
▶              yeni.append(digerleri / (len(c) - 1))
8          c = yeni
9          gecmis.append(c)
10     return gecmis
```

DEĞİŞKENLER

`t = 4` `c = [0, 4.0, 4.0, 4.0]` `digerleri = 7.9` `yeni = [0, 2.6, 2.6, 2.6]`

Kod İzleme: `kosullu_katki(c, tur)`



tur 5: katkılar
[0, 2.6, 2.6, 2.6]

kazanç u = 20 − c + 0,4·Σc
O1: 23,2 O2: 20,5

Tur 5: katkılar [0, 2.6, 2.6, 2.6].

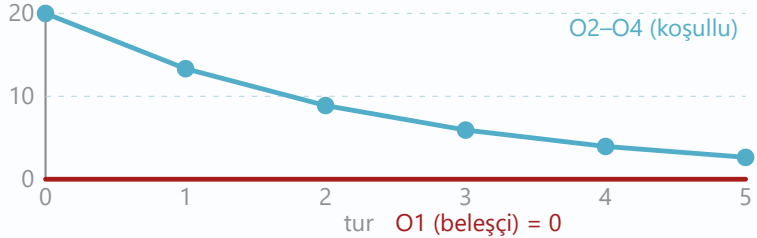
Adım 11 / 12

```
1  def kosullu_katki(c, tur):
2    gecmis = [c]
3    for t in range(tur):
4      yeni = [0]                    # O1 hep beleşçi
5      for i in range(1, len(c)):
6        digerleri = sum(c) - c[i]
7        yeni.append(digerleri / (len(c) - 1))
▶      c = yeni
▶      gecmis.append(c)
10   return gecmis
```

DEĞİŞKENLER

`t = 4` `c = [0, 2.6, 2.6, 2.6]` `gecmis = 6 tur`

Kod İzleme: `kosullu_katki(c, tur)`



tur 5: katkılar

[0, 2.6, 2.6, 2.6]

kazanç u = 20 − c + 0,4·Σc

O1: 23,2 O2: 20,5

5 turda katkı 20 → 2,6: tek beleşçi, çoğunluğun işbirliğini çözer (laboratuvardaki düşüşün basit modeli).

Adım 12 / 12

```
1 def kosullu_katki(c, tur):
2   gecmis = [c]
3   for t in range(tur):
4     yeni = [0]                    # O1 hep beleşçi
5     for i in range(1, len(c)):
6       digerleri = sum(c) - c[i]
7       yeni.append(digerleri / (len(c) - 1))
8     c = yeni
9     gecmis.append(c)
▶   return gecmis
```

DEĞİŞKENLER

`gecmis` = **6 tur**

Özet

Ajan ve MAS

Otonom, reaktif, proaktif ve sosyal ajanlar; işbirlikçi, rekabetçi ya da karma sistemler; çoğu gerçek sistem hibrit kontrol kullanır.

Sözleşme ağı

Duyuru → teklif → ödül. Dağıtık ve basit, ama açgözlü: örnekte 18 dk vs optimum 13 dk.

Koalisyon oyunları

v(S) koalisyon değerini verir. Shapley = ortalama marjinal katkı: (30, 40, 50). Ağırlıklı oylamada güç ≠ ağırlık.

Çekirdek

Hiçbir koalisyonun ayrılmak istemediği paylaşımlar; boş olabilir (çoğunluk oyunu).

Kamu malı

r/n < 1 ise katkı vermemek baskın: denge 20, optimum 32 (kişi başı).

Çözümler

Ceza, itibar, eşik ve mekanizma tasarımı teşvikleri sosyal refahla hizalar.

ÖZET

Etkileşimli Oyunlar

Shapley Koalisyon Oyunu

Karakteristik fonksiyondan marjinal katkılar ve Shapley değeri.

Kamu Malı Oyunu

Turlar boyunca katkı; beleşçilerin ve cezanın etkisi.

Dama

İki ajanlı rekabet: 2007'de çözüldü, mükemmel oyunda sonuç beraberlik (Schaeffer vd.).

ÖZET

Örnek Problemler

1. Sözleşme ağı örneğinde görevler G3, G2, G1 sırasıyla ihaleye çıksaydı atama ve toplam süre ne olurdu?
2. Eldiven oyunu: A'nın sol, B ve C'nin sağ eldiveni var; bir çift 1 değerinde. v'yi yazın, Shapley değerini hesaplayın.
3. Aynı eldiven oyununun çekirdeği nedir?
4. Ana örnekte (40, 40, 40) çekirdekte mi? (20, 25, 75) çekirdekte mi?
5. Kamu malı: n = 5, e = 10, r = 2. Herkes tam katkı verirse ve biri beleşçi olursa kazançlar?
6. Kamu malı oyununda r ≥ n olsaydı baskın strateji ne olurdu?

▶ **Cevapları göster**

Kaynaklar

- M. Wooldridge, *An Introduction to MultiAgent Systems*, 2. baskı, Wiley, 2009.
- Y. Shoham, K. Leyton-Brown, *Multiagent Systems: Algorithmic, Game-Theoretic, and Logical Foundations*, Cambridge University Press, 2009 (Bölüm 12).
- G. Chalkiadakis, E. Elkind, M. Wooldridge, *Computational Aspects of Cooperative Game Theory*, Morgan & Claypool, 2011.
- R. G. Smith, "The contract net protocol: High-level communication and control in a distributed problem solver", *IEEE Trans. Computers* C-29(12), 1980.
- L. S. Shapley, "A value for n-person games", *Contributions to the Theory of Games II*, Princeton, 1953; "Cores of convex games", *Int. J. Game Theory* 1, 1971; L. S. Shapley, M. Shubik, *APSR* 48(3), 1954.
- J. O. Ledyard, "Public goods: A survey of experimental research", *Handbook of Experimental Economics*, 1995.
- E. Fehr, S. Gächter, "Cooperation and punishment in public goods experiments", *American Economic Review* 90(4), 2000; U. Fischbacher, S. Gächter, E. Fehr, "Are people conditionally cooperative?", *Economics Letters* 71(3), 2001.
- T. Sandholm vd., "Coalition structure generation with worst case guarantees", *Artificial Intelligence* 111, 1999.

ÖZET

Kaynaklar: Kitaplar, Dersler ve Araçlar

- A. R. Karlin, Y. Peres, *Game Theory, Alive*, American Mathematical Society, 2017 (koalisyonlar ve Shapley değeri bölümü).
- N. Nisan, T. Roughgarden, É. Tardos, V. Vazirani (ed.), *Algorithmic Game Theory*, Cambridge University Press, 2007 (Bölüm 15: maliyet paylaşımı, çekirdek ve Shapley değeri).
- C. W. Reynolds, "Flocks, herds and schools: A distributed behavioral model", *SIGGRAPH '87*, 1987; T. C. Schelling, "Dynamic models of segregation", *J. Mathematical Sociology* 1(2), 1971.
- FIPA, *Contract Net Interaction Protocol Specification* (SC00029H), 2002.
- Ders sayfaları: N. Shah, CSC304 (Toronto) · A. Procaccia, 15-896 (CMU) · T. Roughgarden, CS364A (Stanford) · M. Hoefer, AGT (Frankfurt)
- Araçlar: Nashpy (iki oyunculu oyunlarda Nash dengesi; ör. çıkar yelpazesindeki üç oyun) · Gambit (açık kaynak oyun kuramı araçları).

Son

Bölüm 7: Çoklu Ajan Sistemleri

Sonraki bölüm: Bölüm 8: Ağ Tasarımı ve Yönlendirme

Çalışma soruları · Terim sözlüğü